AI-ENABLED COCONUT LAND UTILIZATION, WCLWD SEVERITY ASSESSMENT, GERMPLASM IDENTIFICATION AND COMMUNITY INTELLIGENCE ADVISORY SYSTEM

COMMON INTEGRATED SOLUTION REPORT

Group ID: R26-IT-019

B.Sc. (Hons) Degree in Information Technology

Specialising in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

Team members

Team member	Student ID	Individual component
Isuri Darshika Harischandra	IT22549204	Component 4 – AI-Based Coconut Disease Image Classification and RAG Advisory Chatbot
Malshani W.A.M.S.	IT22088314	Component 2 – WCLWD Classification, Explainability, Weakly Supervised Segmentation and Severity Assessment
Wijerathna W.M.H.N.	IT22904164	Component 3 – Aerial Land-Utilization and Approximate Planting-Capacity Analysis
Randeniya A. A. S. L. B. R. P. W. R. C	IT22236500	Component 1 – AI-Based Image Classification for Coconut Germplasm Identification

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. We declare that this common report is our own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of our knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. We grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this report, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. We have disclosed all use of generative AI and other automated tools in accordance with the module policy. We remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this report.

Student name	**Student ID**	**Signature**	**Date**
Harischandra R.D.I.D.	IT22549204		
Malshani W.A.M.S.	IT22088314		
Wijerathna W.M.H.N.	IT22904164		
Randeniya A. A. S. L. B. R. P. W. R. C	IT22236500		

Supervisor certification. The above candidates carried out this research project under my supervision.

Role and name	**Signature**	**Date**
Supervisor: Prof. Sanvitha Kasthuriarchchi		
Co-supervisor: Ms. Karthiga Rajendran		
External supervisor: Mr. Oshan Thilakarathne (CRI)		

ABSTRACT

Coconut (Cocos nucifera L.) is a key estate crop in Sri Lanka that provides household income, employment in the plantation sector and export earnings for processed coconut products. The problem, however, is that the management of coconut resources in the country is disjointed in four parallel information needs: the identification of germplasm within a living collection, assessing land productivity of estates, the state of Weligama Coconut Leaf Wilt Disease (WCLWD) and other foliar damages, and providing industry stakeholders with timely need-based, evidence-based advisory information. Existing activities in these four areas are supported by patchy field visits, manual record keeping and expert opinion.

This common integrated report presents the AI-Enabled Coconut Land Utilization, WCLWD Severity Assessment, Germplasm Identification and Community Intelligence Advisory System, a four-component research platform delivered by Group R26-IT-019. Component 1 is a germplasm classification module that combines an Image Net-pretrained Efficient Net-B0 feature extractor with a Prototypical Network and Grad-CAM visualisation to classify Dwarf, King Coconut and Tall images. Component 2 is a WCLWD severity-assessment module that fine-tunes Efficient Net-B0 for four leaf-condition classes, refines Grad-CAM activations into weakly supervised pseudo-masks and computes an ABCD-inspired Disease Severity Index. Component 3 is an aerial land-utilization and approximate planting-capacity module that fine-tunes a YOLO11n detector for Palm/Tree instances and chains Excess Green vegetation masking, Euclidean Distance Transform clearance analysis and a greedy spacing-aware placement procedure. Component 4 is a disease identification and advisory module built around a second Efficient Net-B0 classifier for four foliar categories, a Mahalanobis-distance out-of-distribution check and a bilingual retrieval-augmented generation chatbot indexed over sixteen domain documents.

The four components are integrated behind a common web platform (React frontend, Node.js/Express application backend, Flask python inference services, MySQL persistence) that shares standards for image pre-processing, model and config

versioning, communicating uncertainty, and responsibly releasing experimentation outputs. The component-wise output measures are: 96.53% test set accuracy (on germplasm validation images) for germplasm classification, 98.79% test set accuracy (on WCLWD validation images) for WCLWD classification, 0.835 mAP@50 (mean average precision@50) for aerial detection of Palm/Tree, and 98.89% test accuracy for four-class disease classifier, along with demonstrations of pseudo-mask creation, severity scoring, free-space calculation, out-of-distribution detection, answer generation with retrieval augmented model, and other outputs. The common thread is to produce transparent, end-to-end decision support that generates inspectable intermediate results and decouples new model metrics from empirically unverified real-world performance. Independent field assessment, expert review, and application-specific metric alignment are all clearly ahead before these combined outputs inform real decisions.

Keywords: coconut management, EfficientNet-B0, YOLO11n, Prototypical Network, retrieval-augmented generation, integrated AI platform.

ACKNOWLEDGEMENTS

We sincerely thank our research supervisor Prof. Sanvitha Kasthuriarchchi, and our co-supervisor Ms. Karthiga Rajendran, for their continuous guidance, constructive feedback, and scholarly assistance in progress during the course of this integrated project. We also thank our external supervisor Mr. Oshan Thilakarathne, CRI, Sri Lanka for his domain information on coconut genetic resources, Weligama coconut leaf wilt disease and plantation management practices which greatly influenced the direction of the four research components.

We are grateful to the Sri Lanka Institute of Information Technology for enabling our work by providing the academic setting, computer equipment, library resources, and infrastructure for managing the project. We thank the authors of the public datasets used in this project - the Kaggle Coconut Diseases and Pest Infestations dataset and the Aerial Images of Palm Trees dataset released in conjunction with Ammar, Koubaa, and Benjdira's palm-counting work - without which the data-driven aspects of this integrated solution could not be built.

We thank Mr. Samitha Vidhanaarachchi for the technical support and advise on deep learning to develop this implementation on coconut imagery and to the authors of the open-source scientific-Python, PyTorch, Ultralytics YOLO, timm, Pinecone, BAAI BGE, Sentence-Transformers, OpenCV and React/Node.js/Flask ecosystems for which this implementation depends. We thank our families, friends and fellow students for the ongoing encouragement during our research and thesis writing.

TABLE OF CONTENTS

1 INTRODUCTION AND PROJECT CONTEXT _____ 1
1.1 Background and context _____ 1
1.2 Stakeholders and user needs _____ 3
1.3 Alignment with the Sustainable Development Goals _____ 5
1.4 Purpose and scope of the common report _____ 6
2 OVERALL PROBLEM DEFINITION AND FOUR SUBPROBLEMS _____ 8
2.1 Overall problem definition _____ 8
2.2 Root causes and overall research gap _____ 10
2.3 Subproblem and component mapping _____ 12
2.4 Overall aim and group objectives _____ 15
2.5 Scope assumptions and constraints _____ 17
3 OVERALL SOLUTION STRUCTURE _____ 20
3.1 Integrated solution overview _____ 20
3.2 High level architecture _____ 22
3.3 Component 1 summary – Germplasm Identification _____ 24
3.4 Component 2 summary – WCLWD Severity Assessment _____ 27
3.5 Component 3 summary – Aerial Land Utilization _____ 30
3.6 Component 4 summary – Disease Identification and Advisory _____ 34
3.7 End to end user and data flow _____ 38
4 COMPONENT INTEGRATION AND COHESIVE OPERATION _____ 41
4.1 Integration strategy _____ 41
4.2 Interfaces, data contracts and shared models _____ 44
4.3 Shared non-functional requirements _____ 48
4.4 Integrated workflow and exception handling _____ 51
4.5 Team contribution and dependency matrix _____ 54
5 INTEGRATED IMPLEMENTATION AND PROJECT EXECUTION _____ 57
5.1 Common technology stack and infrastructure _____ 57
5.2 Development and integration process _____ 61
5.3 Deployment and operational configuration _____ 65
5.4 Evidence of team execution _____ 68
6 INTEGRATED TESTING EVALUATION AND VALIDATION _____ 71
6.1 Evaluation framework and success criteria _____ 71
6.2 Integration and end to end testing _____ 74

6.3 Overall results 77
6.4 Stakeholder and user validation 82
6.5 Achievement of common objectives 85
6.6 Integrated limitations and threats 88
7 COMMERCIALISATION, SUSTAINABILITY AND RESPONSIBLE INNOVATION 92
7.1 Value proposition and beneficiaries 92
7.2 Market alternatives and differentiation 95
7.3 Business model and feasibility 97
7.4 Adoption, intellectual property and compliance 99
7.5 Sustainable Development Goal contribution 101
8 CONCLUSION AND FUTURE DIRECTIONS 104
8.1 Overall conclusion 104
8.2 Principal integrated contribution 106
8.3 Future directions 108
REFERENCES 111
APPENDICES 115

LIST OF FIGURES

Figure 3.1: High-level architecture of the integrated coconut-management platform
Figure 3.2: Germplasm identification component (Component 1) workflow
Figure 3.3: WCLWD severity assessment component (Component 2) workflow
Figure 3.4: Aerial land-utilization component (Component 3) processing stages
Figure 3.5: Disease identification and advisory component (Component 4) workflow
Figure 3.6: End-to-end user and data flow across the integrated platform
Figure 4.1: Interface topology between the four components and shared services
Figure 4.2: Integrated exception-handling and fallback paths
Figure 5.1: Deployment view of the integrated platform
Figure 5.2: Group version-control and sprint cadence summary
Figure 6.1: Confusion matrix for the germplasm classifier (50 validation episodes)
Figure 6.2: Per-class validation metrics of the aerial Palm/Tree detector
Figure 6.3: Grad-CAM, pseudo-mask and DSI illustration (Component 2)
Figure 6.4: Baseline circular-buffer utilization heat map (Component 3)
Figure 6.5: Refined free-space map with ten highest-clearance pockets (Component 3)
Figure 6.6: Greedy capacity estimate – 55 proposed positions at 148.2-px spacing proxy
Figure 6.7: Example RAG chatbot session with retrieved context (Component 4)

LIST OF TABLES

Table 2.1: Summary of the four subproblems and the components addressing them
Table 2.2: Group objectives mapped to components, evidence and acceptance criteria
Table 3.1: Component owners, inputs, methods and outputs
Table 4.1: API endpoints exposed by the four Flask services
Table 4.2: Shared data contracts and persisted entities
Table 4.3: Shared non-functional requirements and acceptance criteria
Table 4.4: Team contribution and dependency matrix
Table 5.1: Common technology stack across the platform
Table 5.2: Development, review and release conventions
Table 6.1: Integration test matrix and observed results
Table 6.2: Overall component-level results summary
Table 6.3: Per-class validation report for the aerial Palm/Tree detector
Table 6.4: Per-class test report for the WCLWD classifier
Table 6.5: Per-class validation report for the germplasm classifier
Table 6.6: Per-class test report for the disease classifier
Table 6.7: Achievement of group objectives
Table 7.1: Value proposition by beneficiary
Table 7.2: Alignment with Sustainable Development Goals

LIST OF ABBREVIATIONS

Abbreviation	Meaning
ABCD	Asymmetry, Border irregularity, Colour variation and Diameter/distribution-inspired features
AI	Artificial Intelligence
AMP	Automatic Mixed Precision
AP	Average Precision
API	Application Programming Interface
AUROC	Area Under the Receiver Operating Characteristic curve
BGE	BAAI General Embedding model family
CCI	Coconut Caterpillar Infestation
CLAHE	Contrast Limited Adaptive Histogram Equalization
CNN	Convolutional Neural Network
COCO	Common Objects in Context (image dataset)
COGENT	International Coconut Genetic Resources Network
CRI	Coconut Research Institute of Sri Lanka
CSV	Comma-Separated Values
CUDA	Compute Unified Device Architecture
DSI	Disease Severity Index
EDT	Euclidean Distance Transform
ExG	Excess Green vegetation index
F1	Harmonic mean of precision and recall
FN	False Negative
FP	False Positive
GPU	Graphics Processing Unit
Grad-CAM	Gradient-weighted Class Activation Mapping
GSD	Ground Sampling Distance
ID / OOD	In-distribution / Out-of-distribution
IoU	Intersection over Union
JSON	JavaScript Object Notation
KDE	Kernel Density Estimation
LAB	CIE L*a*b* colour space
LLM	Large Language Model
mAP	mean Average Precision
MVP	Minimum Viable Product
ONNX	Open Neural Network Exchange
PCA	Principal Component Analysis
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
ReLU	Rectified Linear Unit
REST	Representational State Transfer
RGB	Red, Green and Blue
RQ	Research Question
SDG	Sustainable Development Goal
SLIIT	Sri Lanka Institute of Information Technology
TP	True Positive
UAV	Unmanned Aerial Vehicle
WCLWD	Weligama Coconut Leaf Wilt Disease

WSSS	Weakly Supervised Semantic Segmentation
YOLO	You Only Look Once

1 INTRODUCTION AND PROJECT CONTEXT

1.1 Background and context

Coconut (Cocos nucifera L.) is a plantation crop with long-established economic, cultural and nutritional significance in Sri Lanka. Coconuts provide livelihoods for a large smallholder sector, nutritional and food security through fresh coconut, coconut water and copra oil, and liquid and solid bio-based by-products for export markets as desiccated coconut, coconut oil, activated carbon and coir-based products. The Coconut Research Institute of Sri Lanka (CRI) holds over 100 local and exotic germplasm collections, and conducts breeding and agronomic research, pest and disease management, and genetic resource conservation. Globally, the International Coconut Genetic Resources Network (COGENT) has been calling for the systematic collection, characterisation and sustainable use of coconut germplasm and related genetic resources for a long time.

In practice, managing the resources of a plantation involve attempting to keep pace with a set of concurrent and overlapping information demands. Where are cultivars or morphological groups located in a certain field? How fully has the land been populated with mature trees? Is a certain cultivar or group currently displaying the symptoms of a disease or pests? What does the latest research tell us about handling this issue? In the status quo scenario, these questions are generally answered with separate visual surveys, spreadsheets, paper records, informal consultation with experts and haphazard searches for information in publications. Fragmented workflows hinder the ability to develop a unified view of the health and productivity of a plantation over time and limit the potential for comparisons.

Weligama Coconut Leaf Wilt Disease (WCLWD) - a phytoplasma related syndrome that was first identified in the low-country wet zone of the south. WCLWD has similar symptoms to various other biotic and abiotic stresses, and molecular testing using methods such as nested PCR is necessary for confirmation and such tests are not available on site. A similar pattern exists for other problems such as coconut caterpillar infestation (Opisina arenosella), Gray Leaf Spot and Leaf Rot where the symptoms are the same but image-based decision support is useful in conjunction

with the final diagnosis of an expert. Managing the land itself is also complicated since perennial palms can dominate a planting site for decades, making decisions on replacing missing accessions, introducing new germplasm or reconfiguring a field dependent on accurate information on canopy occupancy and the availability of land that can be planted.

The AI-Enabled Coconut Land Utilization, WCLWD Severity Assessment, Germplasm Identification and Community Intelligence Advisory System focuses specifically on these dispersed tasks. The project puts together four autonomous but interconnected research tasks behind the umbrella of a web-based application, so that one unified system can provide on the same user interface germplasm identification, land utilization analysis, disease severity assessment, and bilingual advisory services. Each task is solved with a deep-learning or image-processing solution appropriate for the subproblem, with an emphasis on explainability, communicating uncertainty, and honest differentiation between measured model performance and untested real-world deployment.

1.2 Stakeholders and user needs

Based on these principles and finding a way to include them in the design of the integrated solution was an important part of the process. The integrated solution was based on a preliminary set of primary and secondary stakeholders as follows: primary stakeholders: coconut growers (smallholder producers and estate growers), coconut research scientists at CRI and other institutions, Agricultural Extension Officers providing farmer advisory services at the field level, Students and other technical staff engaged in germplasm characterisation and conservation of coconut. Secondary stakeholders: Coconut sector policy makers, Drone services providers providing imagery of farms, a systems developer and the wider Coconut research community which receives images for comparison.

These are summarised here. From stakeholder consultations, informal interviews and review of the literature, growers are given visual aids for identifying three of the major pests and diseases; a variety of picture-based advisory tools with the specific aim of guiding crop decision making; and information on the main planting material

varieties. Growers would also find it helpful to have some indication of the spread of regional planting material before they enter the field. From research scientists, extension officers and all stakeholder groups, there is a need for easily accessible, reproducible data sets on various aspects of plantations, that can be examined over a period of time. Plantation managers are most interested in short listing areas for field visits, recording the current level of utilisation of a particular plot and suggesting suitable plots for further planting. There is also a general desire for outputs that acknowledge uncertainty and, whilst being honest, also provide guidance to help focus the efforts of users.

Table 1.1 indicates the primary stakeholder mappings between their primary information needs and the artefacts within this project which are a response to this need. This map is provided as an aid to understanding rather than as a comprehensive requirements catalogue. Primary stakeholder artefacts are provided in Chapter 3 and end-to-end user workflows are provided in Section 3.7.

Stakeholder	**Primary information need**	**Component(s) addressing the need**
Smallholder growers	Preliminary identification of leaf diseases and plain-language advice	Component 4 (disease classifier + RAG chatbot)
Estate / plantation managers	Snapshot of canopy occupancy, free space and planting opportunities	Component 3 (aerial land utilization)
CRI research scientists	Reproducible variety indication and disease severity records	Components 1, 2 and 4
Extension officers	Shared platform to review images and generate referral notes	All components via the common web application
Field technicians collecting germplasm images	Automated grouping of Dwarf, King Coconut and Tall categories	Component 1 (germplasm identification)
Drone-survey providers	Downstream analysis for aerial RGB mosaics they already capture	Component 3 (aerial land utilization)

1.3 Alignment with the Sustainable Development Goals

There are potential contributions to several UN SDGs. SDG 2 (Zero Hunger) is relevant as the integrated platform enables better germplasm identification and disease monitoring for land-use planning in a food-crop. SDG 9 (Industry, Innovation and Infrastructure) is relevant as the project offers digital innovation, deep learning and interpretable image analysis to a traditional plantation industry. SDG 12 (Responsible Consumption and Production) is relevant as more efficient

utilizing of currently planted land and improving disease management could lead to more efficient use of inputs. SDG 15 (Life on Land) is relevant as better germplasm identification may contribute to sustainable use of plant genetic resources. However, crop yields, use of chemicals, income for farmers and the ecological impacts are not estimated in this report and therefore these links to SDGs are pathways rather than impacts.

1.4 Purpose and scope of the common report

This common integrated solution report describes the complete, integrated AI-Enabled Coconut Land Utilization, WCLWD Severity Assessment, Germplasm Identification and Community Intelligence Advisory System. The report describes the shared problem, the four-component architecture, the integration approach which integrates the components into a single web application, the integrated assessment evidence and the common conclusions concerning value, limitations and work towards deployment. It does not repeat the detailed methodology, experimentation and results of individual research projects, which continue to be the primary sources of evidence at the component level. For each quote of a measurement, configuration constant or technical artefact in this common report, the respective thesis is also cited.

This report targets the project supervisors and the examination panel, CRI external supervisor and those researchers and practitioners who will come across the integrated platform through the dissemination activities of the group. Contents This report is self-contained for the integrated solution and component level detail is left to individual theses in accordance with the module template and the SLIIT documentation standard.

2 OVERALL PROBLEM DEFINITION AND FOUR SUBPROBLEMS

2.1 Overall problem definition

The typical challenge this project seeks to solve is: generating and consuming information for coconut land, germplasm, disease and advisory decisions happens on disconnected, predominantly paper and computer-standalone-input, -analysis workflows. Visual field observations, paper field notebooks, blank computer spreadsheets, casual expert advice are all used to capture information for one particular activity, but none of these captures the whole story to be stored into a single, standard record that can be revisited, compared across locations, combined and interpreted. As a result, those who grow coconut and those who research grow coconut are over-invested in data collection and integration while still lacking quantitative, repeatable baselines for assessing change in coconut.

Equally simply, the project hypothesis is a common problem at the level of the application for which an interpreted decision-support system should address. Such a system should accept normal physical inputs (leaves, aerial RGB) and a natural language question, use AI and image-processing appropriately according to the nature of the information, and produce results that adhere to the four principles above. It's made explicit that the nature of the information-need is being targeted here as distinct from the technologies used to solve the problem: a solution is not defined here as 'deep learning' or 'computer vision' but as any single, human-understandable integrated workflow satisfying the information need defined in Section 1.2. The solution described in this report is one of many.

2.2 Root causes and overall research gap

Multiple interacting factors make this "simple" problem go on. First, these four subproblems - germplasm identification, land-utilization quantification, disease severity estimation and evidence-grounded advice - involve different data modalities, model paradigms and regimes of permissible error. -B - germplasm identification can be tackled with transfer-learning feature extractors on street-level close-up images; -

land use can be handled with object detectors on aerial RGB imagery; -disease severity needs both category classification and spatial localisation; -and advice interaction needs retrieval over curated technical literature and language generation.1no current off-the-shelf tool addresses all four well.

Second, these published studies each solve one of these tasks. Mohanty, Hughes and Salathé [1] describe the use of deep learning to classify plant disease images; Vidhanaarachchi et al. [2] present deep-learning diagnosis and severity assessment of WCLWD and the coconut caterpillar; Tan and Le [3] present the Efficient Net family, used by three of the four components; Lee et al. [4] present the feature-distance framework for out-of-distribution detection, used by Component 4; Lewis et al. [5] present the retrieval-augmented generation paradigm used by the advisory chatbot; Ammar, Koubaa and Benjdira [6] present the deep-learning-based palm counting and geolocation from aerial imagery, providing the dataset used by Component 3. Each is designed to validate a single method for a single use case; they do not describe a combined workflow such as the one this project proposes.

Third, none of the published papers actually close the performance loop to realise the plantation system. Publications therefore report typical metrics on a held-out validation/test subset of the same training dataset, which does not demonstrate how the system generalises to other plantations, camera setups, seasons or disorder variants. Very few papers articulate the calibration coefficients, configuration constants, deployment artefacts or setup that are needed to reproduce the system, and very few examples document whether and how an interactive user might read and interpret the output to determine if evidence accumulation or candidate forward delegation to an expert is warranted. The gap this project aims to fill is therefore the integration gap: to deploy, annotate and evaluate a multi-component platform where interpretability, uncertainty, and operational realism are managed as carefully as the several measures of headline accuracy.

2.3 Subproblem and component mapping

The common problem is decomposed into four non-overlapping but interdependent subproblems, each addressed by one component. The subproblems are listed below and summarised in Table 2.1.

1. Subproblem 1 (Germplasm identification). Given a photograph of a coconut palm or its fruit, produce a preliminary indication of the morphological category (Dwarf, King Coconut or Tall) together with a visual explanation of the regions that influenced the prediction, using a small and imbalanced labelled dataset.
2. Subproblem 2 (WCLWD severity assessment). Given a leaf image, produce a four-class condition prediction (Healthy, WCLWD Drying of Leaflets, WCLWD Flaccidity, WCLWD Yellowing), a visual explanation of the discriminative region, a weakly supervised pseudo-mask of the suspected affected tissue and a computational Disease Severity Index with a severity category.
3. Subproblem 3 (Aerial land utilization and planting-capacity analysis). Given an aerial RGB image of a plantation plot or field gene bank, produce an interpretable estimate of occupied canopy area, available area, high-clearance free-space pockets and a spacing-aware approximate planting-capacity figure.
4. Subproblem 4 (Disease identification and bilingual advisory). Given a leaf image or a natural-language question, produce a four-class disease prediction (Coconut Caterpillar Infestation, Gray Leaf Spot, Healthy Leaves, Leaf Rot) with an out-of-distribution check for unfamiliar inputs, and generate a retrieval-grounded conversational answer in English or Sinhala that draws on a curated knowledge base of sixteen coconut-domain PDFs.

Table 2.1: Summary of the four subproblems and the components addressing them

Subproblem	Component	Owner	Primary input	Primary output
1 – Germplasm identification	Component 1	Randeniya A. A. S. L. B. R. P. W. R. C (IT22236500)	Close-up coconut image (RGB)	Predicted class (Dwarf / King Coconut / Tall) + Grad-CAM overlay
2 – WCLWD severity assessment	Component 2	[Student 2]	Coconut leaf image (RGB)	Four-class prediction + Grad-CAM + pseudo-mask + Disease Severity Index + category
3 – Aerial land	Component 3	Wijerathna	Aerial RGB plot	Detections +

utilization		W.M.H.N.	image	utilisation %, free-space pockets, candidate planting positions
4 – Disease identification + bilingual advisory	Component 4	Isuri Darshika Harischandra (IT22549204)	Leaf image and/or user question (English or Sinhala)	Four-class prediction + OOD flag + retrieved context + chatbot response

These four subproblems are intentionally disjoint at the level of the ML modelling - each subproblem has its own training data, ML architecture and evaluation protocol - whilst being joined up at the user-workflow layer (eg a grower uploading an image of a leaf to the platform will generally derive benefit from both the Component 4 disease prediction and the Component 4 advisory text; a plot-based research scientist will generally make use of the Component 3 utilisation outputs along with the Component 1 variety, origin and cropping data for that plot; and a WCLWD group extension exercise will typically combine the Component 2 severity output and the Component 4 chatbot). The integration layer, discussed in Chapter 4, allows us to build these end-to-end user journeys from composition of component outputs rather than building this journey into a single monolithic ML model.

2.4 Overall aim and group objectives

Aim. To design, implement, integrate and evaluate an AI-enabled coconut management platform that provides preliminary germplasm identification, aerial land-utilization analysis, WCLWD severity assessment and bilingual disease advisory through a single web application, with transparent reporting of model performance, limitations and deployment considerations.

5. Deliver a four-component solution covering germplasm identification, land utilization, WCLWD severity assessment and disease advisory, each with its own trained model and deployment artefacts.
6. Expose the four components through a shared React / Node.js / Flask / MySQL web platform with a consistent user experience and shared conventions for authentication, logging and uncertainty communication.

7. Document interpretable intermediate outputs for each component – Grad-CAM overlays, pseudo-masks, free-space maps, retrieved context passages and OOD scores – so that every prediction can be inspected rather than consumed as an opaque label.
8. Evaluate each component at the measured level reported in its individual thesis and report integrated evaluation evidence at the platform level, including an integration test matrix, response examples and user-workflow walk-throughs.
9. Record the limitations of the current implementation honestly and identify the concrete field, expert and metric-calibration steps required before any of the integrated outputs can be relied on for consequential agronomic decisions.

Table 2.2: Group objectives mapped to components, evidence and acceptance criteria

Group objective	Evidence used for evaluation	Mapped components
Deliver the four components	Trained model artefacts; per-component accuracy, mAP or F1 metrics	C1, C2, C3, C4
Integrate behind the shared platform	Integration test matrix; API response examples; application screenshots	All
Preserve interpretable intermediate outputs	Grad-CAM, pseudo-masks, free-space maps, retrieved context passages, OOD score	C1, C2, C3, C4
Report component- and platform-level evaluation	Component accuracy tables; integration test matrix; walk-through screenshots	All
Document limitations and next steps	Chapter 6.6 and Chapter 8 of this report; individual-thesis limitation sections	All

2.5 Scope assumptions and constraints

Scope.

The scope of the integrated solution is four components and the shared web platform, datasets, models and configuration artefacts described in Chapters 3 to 5. The scope of each component is in the categories of scope presented for each thesis: Dwarf, King Coconut and Tall for Component 1; Healthy Leaves, WCLWD Drying of Leaflets, WCLWD Flaccidity and WCLWD Yellowing for Component 2; Palm and Tree aerial object detection for Component 3; and Coconut Caterpillar Infestation, Gray Leaf Spot, Healthy Leaves and Leaf Rot plus three advisory topics for

Component 4. The solution is not scoped for genetic confirmation, molecular diagnosis, agronomic spacing calibration and field level yield prediction.

Assumptions.

The project makes the following assumption that (i) public datasets used are suitable proxy for proof-of-method evaluation, (ii) user submitting an image to the system agrees to have this processed and displayed on the platform with a transparent level of uncertainty, (iii) exported model / configuration artefacts are internally consistent - weights, class map, calibration constants and pre-processing settings are all versioned together, and (iv) there is hardware resources available to run the language models with a GPU/CPU for inference, and internet connection to the cloud services Pinecone vector index and Open Router language model for the advisory chatbot.

Constraints. The services are hosted on commodity cloud infrastructure, using Python 3.12 for inference, Pytorch 2.x with CUDA for the deep-learning components, Ultralytics YOLO11 for aerial detection, timm for Efficient Net-B0, OpenCV for image processing, React 18 for the frontend, Node.js / Express for the application server, Flask for the Python model servers, and MySQL for persistence. The project is not equipped with mobile-native clients, offline-first operation, or federation across multiple institutional deployments; these are noted as priorities for future work in Chapter 8.

3 OVERALL SOLUTION STRUCTURE

3.1 Integrated solution overview

The integrated product is one web app providing four different workflows - germplasm identification, aerial land use planning, WCLWD severity diagnosis and disease identification plus bilingual advisory - through a shared interface, common login and shared conventions for image pre-processing, versioning and communication of uncertainty. To the user the product provides a home page linking to each workflow, a result inspection view showing the high-level prediction together with the interpretable intermediate artefacts, and a conversational interface for the advisory chatbot. To the operator, the application is a React application served via a Node.js / Express application, which proxies requests to four Flask inference services - one for each component - and saves requests, responses and reference records to a shared MySQL database.

The core offering is "4 analysis types with AI support in one place, which share their interpretable intermediate output instead of leaving only a prediction behind." The final result is "quickly checkable backup when doing routine coconut-related work, without jumping between services, or copying output between them." As one team member puts it, "this is not an automated decision-making tool, but a workspace for decision-making - an environment in which the decision (which is agronomic) is still yours."

3.2 High level architecture

. Figure 3.1 – High level architecture of solution. External users publish to /call /predict of the React app through https. The React app calls the Node.js / Express app through HTTP / HTTPS where it authenticates, validates / sanitizes / audits / logs the request and initiates the corresponding Flask python service call. The Flask service loads a model artefact set (e.g. Trained weights, class id map, calibration or configuration json etc), runs inference and post-processing on the individual uploaded images, and then sends a json response that contains the prediction with a score / confidence along with identifiers to intermediate artefacts such as pseudo

mask PNG, Grad CAM overlay images, free space map images, context passages retrieved from semantic memory. Artifacts are either stored directly in the json (for example as base-64 strings) or stored in shared object store and referenced by URL.

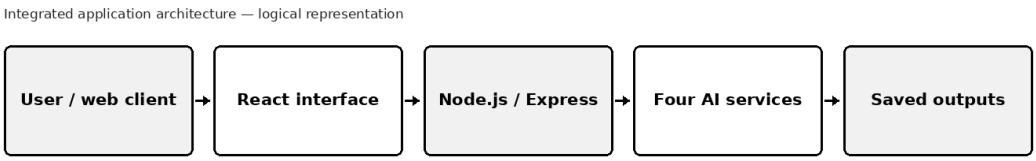


Figure 3.1: High-level architecture of the integrated coconut-management platform. The React frontend communicates with a Node.js / Express application backend, which forwards requests to four Flask inference services (one per component). Shared MySQL persistence holds users, requests, responses and reference knowledge-base metadata. **(schematic illustration).**

The architecture is designed to be modular so that any component can be deployed, swapped out or scaled independently. Shared services: authentication and session management (node.js backend), persistent storage (MySQL), the Pinecone vector index (used by component 4's advisory chatbot), and an object store used to hold large intermediate artefacts. External systems: the language-model endpoint connected to Open Router (used by component 4 to generate the advisory response from retrieved context).

3.3 Component 1 summary – Germplasm Identification

Owner: Randeniya A. A. S. L. B. R. P. W. R. C (IT22236500).

Subproblem: Subproblem 1 (germplasm identification).

Input: a close-up image of coconut.

The core: an Image Net-pretrained Efficient Net-B0 feature extractor with the classifi cation head replaced by adaptive average pooling, followed by a linear projection from 1,280 to 512 features with batch normalisation, all trained in a 3-way, 8-shot, 5-query episodic manner over 40 epochs using the Prototypical Network with scaled cosine similarity. Grad-CAM is attached to the final convolutional block to provide visual explanation. Output: a predicted label for category (Dwarf, King Coconut, Tall), scaled-similarity score vector, with a Grad-CAM heat map and overlay.

Contribution to overall solution: a visual on first pass of coconut variety that other elements lack.

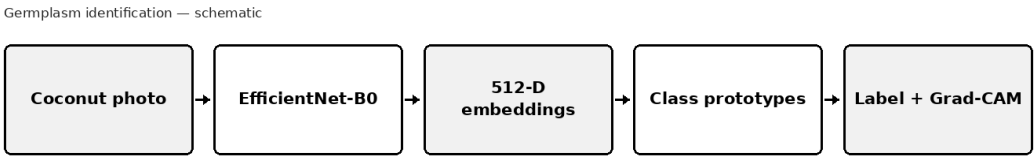


Figure 3.2: Germplasm identification component (Component 1) workflow. An EfficientNet-B0 backbone with a 512-dimensional projection head produces an embedding; the embedding is compared with per-class fixed prototypes using scaled cosine similarity; the top class and its Grad-CAM overlay are returned. **(schematic illustration).**

Component-level evidence (detailed in individual thesis) Per-class F1 of 0.96 (Dwarf), 0.99 (King Coconut) and 0.95 (Tall) 96.53%4 episodic validation accuracy on 50 three-way episodes from an 87-image validation partitionTraining:validation=40:10 converged through training. Encoder exported successfully into Torch Script and ONNX formats The individual thesis specifically notes that the 750 query pred

3.4 Component 2 summary – WCLWD Severity Assessment

Owner: Malshani W.A.M.S. (IT22088314).

Subproblem: Subproblem 2 (WCLWD severity assessment).

Input: a coconut leaf image.

Main approach: a Image Net-pretrained Efficient Net-B0 classifier (Healthy Leaves, WCLWD Drying of Leaflets, WCLWD Flaccidity, WCLWD Yellowing) fine-tuned for 20 epochs using AdamW, cross-entropy loss and a cosine-annealing scheduler on a group-split subset of the public Coconut Diseases and Pest Infestations dataset. For visual explanation, Grad-CAM overlays were generated and the activation map is converted into a pseudo-mask by means of image-specific Otsu thresholding, Grab Cut, followed by morphological cleanup. An ABCD-inspired Disease Severity Index was then calculated based on asymmetry, border irregularity, colour variation, spatial distribution and percentage of affected area, and turned into a 0-100 severity score and a Mild / Moderate / Severe / Critical severity category. Output: a four-class

prediction and its probability, a Grad-CAM overlay, a pseudo-mask, the DSI and a severity category. How it contributes to the overall solution: the WCLWD severity prediction view that feeds into Component 4's holistic disease classifier.

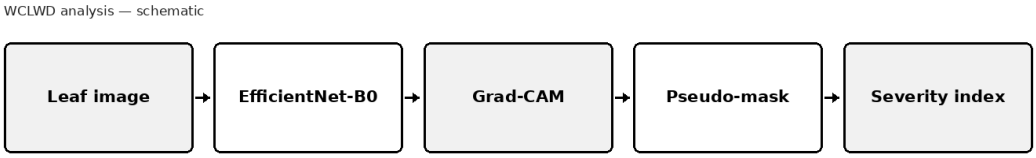


Figure 3.3: WCLWD severity assessment component (Component 2) workflow. An EfficientNet-B0 classifier produces a condition prediction; Grad-CAM is converted to a pseudo-mask through Otsu + GrabCut; ABCD-inspired descriptors and affected-area percentage are combined into a 0–100 Disease Severity Index with a four-level category. **(schematic illustration).**

The reported component-level evidence includes 99.21% best validation accuracy at epoch 18 and 98.79% held-out test accuracy on a 497-image test partition after perceptual-hash based group splitting (2,351 training, 506 validation, 497 test images from 3,354 deduplicated images organised in 3,238 near-duplicate groups). Weighted test precision, recall and F1-score are 98.80%, 98.79% and 98.77%. The individual thesis explicitly flags that no pixel-level ground-truth masks or expert severity ratings were available, so the pseudo-mask and the DSI are computational outputs that require agronomic validation before they can be used in consequential decisions.

3.5 Component 3 summary – Land Utilization Analysis System For Coconut Genebanks

Owner: Wijerathna W.M.H.N. (IT22904164).

Subtask: subtask 3 (air land use and planting capacity analysis)Input: The aerial RGB image of the trial or field gene bank.

Core approach: a four-step image-processing pipeline - (i) fine-tuned YOLO11n object detector for Palm/Tree instances (640 × 640), (ii) refined vegetation mask, infusing an Excess Green vegetation index with elliptical constrains and morphological opening/closing operations within detection buffers, (iii) Euclidean Distance Transform and local-maximum detection of the negative mask to prioritise free-space pockets, and (iv) a greedy, spacing-aware placement routine proposing

further planting points, subject to an image-space minimum spacing. Output: detected palm/tree count, baseline (circular-buffer) and refined utilisation scores, free-space pocket list with clearance values and 'greedy' planting point suggestions. Contribution to the complete solution: land-level occupancy perspective to complement the leaf-level points processed by the other components.

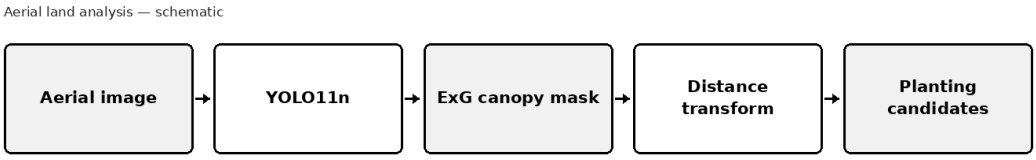


Figure 3.4: Aerial land-utilization component (Component 3) processing stages. YOLO11n detection → Excess Green vegetation mask with ellipse constraint and morphology → Euclidean Distance Transform and local-maximum free-space ranking → greedy spacing-aware capacity estimation. **(schematic illustration).**

The recorded component-level indication is based on the best YOLO11n checkpoints at epoch 39 (overall precision 0.782, recall 0.824, mAP@50 0.835, mAP@50-95 0.489), on the 42-image validation set that produces 1,616 boxes (1,357 Palm, 259 Tree) from the 279-image training set that produces 10,398 total bounding boxes. On the 4,864 × 3,648 demonstration image the detector identified 86 palm/tree instances, the circular-buffer baseline had 32.29% utilisation, the refined ExG mask gave 23.81% utilisation, the EDT + greedy procedure gave 172 candidate pockets and 55 potential additional planting sites at the 148.2 pixel proxy spacing. The individual thesis clearly notes that the dataset is not a coconut-specific site, that all spatial measurements are in pixels, and that the fig. 2 capacity of 55 planting positions is a technical proof-of-concept demonstration rather than an agronomical, calibrated planting plan.

3.6 Component 4 summary – Disease Identification and Advisory

Owner: Isuri Darshika Harischandra (IT22549204).

Subproblem: Subproblem 4 (disease identification and bilingual advisory).

Inputs: a coconut leaf image, a natural-language question in English or Sinhala or both.

An image classifier based on Efficient Net B0 pretrained on Image Net that has a custom head that projects 1280 features to a 256-d vector followed by a set of 4 class logits (CCICaterpillars, Gray Leaf Spot, HealthyLeaves, Leaf Rot); trained using a two-phase transfer-learning approach with early stopping; (ii) a feature-distance out-of-distribution detection based on class means and shared covariance and Mahalanobis distance; threshold set at 99th percentile of validation data in-distribution OOD scores(594.53); and (iii) a retrieval-augmented generation chat-bot extracts text from 16 curated PDFs, broken up into ~1,606 900-character overlapping chunks with 150-character overlaps, embeds with BAAI /bge-large-en-v1.5, stores in Pinecone by cosine similarity, and queries before passing returned passages into an Open Router tied large language model. Outputs: 4-class image prediction with confidence, OOD distance and OOD label, English/Sinhala language chat-bot reply, and list of context chunks retrieved. Contributes to the overall solution: the foliar-disease broad-view, and the text-based, open-ended conversational 'advice desk' view that supports the primary WCLWD severity view in Component 2 and categorical variety view in Component 1.



Figure 3.5: Disease identification and advisory component (Component 4) workflow. An EfficientNet-B0 image classifier with a Mahalanobis-distance OOD check sits alongside a RAG chatbot that retrieves from 1,606 chunks indexed over sixteen PDFs and generates bilingual responses via an OpenRouter-connected LLM. **(schematic illustration).**

There was component-level evidence of 98.89 % test accuracy, 0.3841 test loss on 723 test images, over a class-level split of 70/15/15 (total 4,807) images, with class-level train/val/test counts: 693/148/149 CCICaterpillars, 1,494/320/321 Gray Leaf Spot, 86/18/19 HealthyLeaves, 1,091/234/234 Leaf Rot. Training was augmented and balanced to 2,000 per class, giving a total of 8,000 training examples. The OOD mechanism flagged 0.97% of in-distribution test examples at the chosen threshold. The RAG pipeline successfully ingested 16 PDFs into 1,606 chunks, with interactive

English and Sinhala responses with retrieved context. The individual thesis explicitly flags that no representative external OOD benchmark and no formal chatbot answer-quality benchmark are documented, that the Healthy_Leaves class has only 19 test images, and that the chatbot's three-topic scope (CCI, Gray Leaf Spot and Leaf Rot) should not be presented as an exhaustive advisory service.

3.7 End to end user and data flow

A typical end-to-end user journey through the integrated platform is: A registered user logged into the React frontend chooses a workflow - e.g., Analyse an aerial image of my plot - and uploads an image or inputs a question The React frontend posts the request to the Node.js backend - the request is validated, an audit record is written to MySQL and the request is forwarded to the relevant Flask inference service The Flask inference service loads the relevant model artefact set from the versioned artefact store, applies the component's preprocessing, inference and returns a structured JSON response to the Flask backend The Flask backend writes a response record and passes the payload back to the React frontend where the prediction, confidence or score and component's interpretable intermediate artefacts are rendered to the user For the Component 4 chatbot the user has the option of asking follow-up questions, each of which results in a retrieval step against the Pinecone index before a response is generated by the Open Router language model.

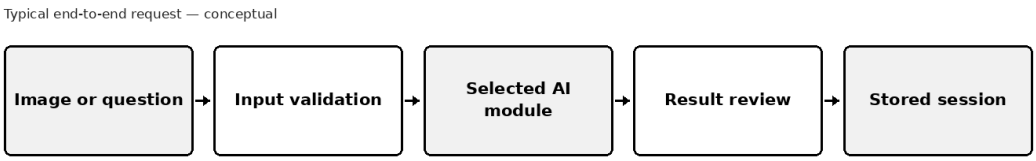


Figure 3.6: End-to-end user and data flow across the integrated platform. A single user request moves from the React frontend through the Node.js backend to the appropriate Flask service, with persistence and shared services (MySQL, Pinecone, object store, LLM endpoint) accessed at well-defined points. **(schematic illustration).**

This flow is designed in a way that the four components can share user context and image upload history and preference settings without sharing an ML model. A single uploaded image can be sent independently to Component 1 (variety indication),

Component 2 (WCLWD severity), and Component 4 (broader disease classification); the frontend can assemble the three results in a single inspection view, displaying each component's confidence and limitation statements individually so as not to convey any implicit consensus among the models. Aerial RGB images are sent only to Component 3, which processes larger inputs, writes larger artefacts (annotated image, free-space overlays, CSV coordinates) to the object store (rather than inline with the response).

Table 3.1: Component owners, inputs, methods and outputs

Component	Owner	Input	Core method	Primary output	Contribution
1 – Germplasm	Randeniya A. A. S. L. B. R. P. W. R. C	Close-up coconut image	EfficientNet-B0 + Prototypical Network + Grad-CAM	Variety class + Grad-CAM	First-pass variety indication
2 – WCLWD	[Student 2]	Leaf image	EfficientNet-B0 + Grad-CAM + Otsu/GrabCut + ABCD DSI	Condition class + mask + DSI	WCLWD-focused severity view
3 – Aerial	Wijerathna W.M.H.N.	Aerial RGB plot image	YOLO11n + ExG + EDT + greedy placement	Tree count + utilisation % + proposed positions	Land-level occupancy view
4 – Disease + advisory	Isuri Darshika Harischandra	Leaf image and/or NL question	EfficientNet-B0 + Mahalanobis OOD + RAG chatbot	Disease class + OOD + chatbot answer	Broader disease + bilingual advice

4 COMPONENT INTEGRATION AND COHESIVE OPERATION

4.1 Integration strategy

The group's approach to system integration is to compose these four elements at the application layer, through clearly specified HTTP APIs and common persistent storage, rather than at the machine-learning level. We make this choice intentionally: each of the four elements is solving its own subproblem, with vastly different data modalities and evaluation metrics and no meaningful benefit to forcing them into a single shared model. By composing at the application level, we keep them independent, enabling the ability to iterate on or replace any one of them without requiring others to be retrained, and to work on a different schedule. At the same time, the actual unit of composition is the end-to-end user workflow, which matches the perspective that users have of the platform.

The integration was performed in 3 overlapping stages: Interface-first; the team agreed on a request / response envelope (Upload identifier, component identifier, prediction, confidence, optional intermediate artefacts, optional error block), and on a common MySQL schema for users, requests, responses, and knowledge-base metadata. All component owners implemented their Flask inference services following this shared contract, while the React frontend was built in parallel. Per-component; each individual inference service implementation was wired into the main Node.js server sequentially, with a new integration test exercising each endpoint. End-to-end; the user journeys as outlined in Section 3.7 were exercised on the complete platform (without log in), as new release candidate platform versions.

Management of integration risk occurred in four ways:

1. We used a shared integration branch in Git with requirement for pull-request review so that no change could pass to the integration environment without review and approval by at least one other contributor.

2. We version controlled our shared contract document and made sure any breaking change was accompanied by a migration note.

3. All Flask services published a /healthz endpoint that the backend used to determine if components had degraded or failed and to surface the error gracefully to the user.

4. We versioned together each component's set of model artefact files as a tagged bundle so the inference service could not be deployed with an inconsistent set of weights, class mapping, calibration file and preprocessing configuration.

4.2 Interfaces, data contracts and shared models

Each component exposes a small set of REST endpoints behind a shared /api/v1/<component>/* prefix. Table 4.1 summarises the endpoints exposed by each Flask service. Table 4.2 summarises the shared data contracts persisted in MySQL.

Table 4.1: API endpoints exposed by the four Flask services

Component	Method	Path	Purpose
C1 Germplasm	POST	/api/v1/germplasm/predict	Upload image → predicted variety + Grad-CAM
C1 Germplasm	GET	/api/v1/germplasm/healthz	Service health check
C2 WCLWD	POST	/api/v1/wclwd/assess	Upload leaf image → condition class + Grad-CAM + pseudo-mask + DSI
C2 WCLWD	GET	/api/v1/wclwd/healthz	Service health check
C3 Aerial	POST	/api/v1/aerial/analyze	Upload aerial RGB → detections + utilisation + capacity
C3 Aerial	GET	/api/v1/aerial/pockets/{request_id}	Fetch ranked free-space pockets for a prior request
C3 Aerial	GET	/api/v1/aerial/healthz	Service health check
C4 Disease	POST	/api/v1/disease/classify	Upload leaf image → disease class + OOD distance + flag
C4 Advisory	POST	/api/v1/advisory/ask	Submit question (optional conversation id) → retrieved context + response
C4 Disease	GET	/api/v1/disease/healthz	Service health check

The shared request envelope requires a user identifier, an optional upload identifier and the component-specific payload. The shared response envelope returns the status, a correlation identifier, the primary prediction block, an optional artefact block with URLs or inline base-64 payloads, an optional error block and timing

information. This uniformity means that the frontend can treat the four components as instances of the same abstract interface and that cross-cutting concerns such as retries, timeouts and error reporting are implemented in one place.

Table 4.2: Shared data contracts and persisted entities

Entity	Key fields	Owner service	Purpose
users	user_id (PK), email, role, created_at	Node.js backend	Account management and audit
uploads	upload_id (PK), user_id, blob_url, mime_type, created_at	Node.js backend	Record of uploaded images
requests	request_id (PK), user_id, component, upload_id, payload_json, created_at	Node.js backend	Audit trail of API calls
responses	response_id (PK), request_id, status, prediction_json, artefact_urls, latency_ms, created_at	Node.js backend	Audit trail of model responses
knowledge_sources	source_id (PK), title, authors, year, topic, included_in_rag	Component 4 service	Metadata for the sixteen advisory PDFs
chat_sessions	session_id (PK), user_id, language, started_at, ended_at	Component 4 service	Conversation state for the advisory chatbot
chat_turns	turn_id (PK), session_id, role, text, retrieved_sources_json, created_at	Component 4 service	Per-turn messages and retrieved context

4.3 Shared non-functional requirements

The group agreed on a small set of shared non-functional requirements that apply across all four components. Table 4.3 lists these requirements, the acceptance criteria used during integration testing and the component owners' primary responsibilities for each.

Table 4.3: Shared non-functional requirements and acceptance criteria

Area	Requirement	Acceptance criterion
Performance	Interactive inference latency	Image components return within 5s on reference server at 90th percentile
Performance	Chatbot response time	Advisory service returns within 10s on reference server at 90th percentile
Usability	Consistent uncertainty wording	Every prediction view shows a textual confidence and a limitation note

Security	API-key handling	No model/API secrets in source control; all secrets injected via environment
Security	Uploaded image handling	Images retained only as long as needed to serve the response, per policy
Privacy	User data minimisation	Platform collects only the minimum data needed for the service
Reliability	Graceful service degradation	Backend detects unhealthy services and returns a plain-language error
Maintainability	Model artefact versioning	Each component bundles weights, class map and config as one tagged artefact set
Scalability	Independent service scaling	Each Flask service can be scaled horizontally without changes to others
Accessibility	Keyboard and screen-reader support	Primary frontend flows pass automated accessibility checks
Professional standards	IEEE referencing throughout	All citations in this report follow a single IEEE referencing style

4.4 Integrated workflow and exception handling

The normal integrated workflow for an image-based component is upload - validate-and-persist - infer (via the Flask service) - present (front-end). There is an extra retrieve step for Component 4's advisory service, situated between the validate-and-persist and infer, in which the user's question is embedded, matched to the Pinecone index, and used to build the context fed to the language model. The front-end presents the prediction, the component's score, the interpretable artefacts and- most importantly- the component's limitation statement, which is taken from a shared library of agreed wording to ensure that the four components communicate uncertainty in the same way.

The primary exception paths are: (i) invalid upload - wrong mime type, file too large, corrupt image - handled in the backend at the time of upload with a 400 response and a clear message in the response (ii) unhealthy Flask service - found through regular /healthz checks and surfaced in the response as a 503 with a helpful message that this specific workflow is temporarily unavailable (iii) model-inference failure (wrong shape, missing artefact) caught in the Flask service and returned as a structured error block. (iv) retrieval or language-model failure in Component 4 (falling back to a safe

answer that indicates we do not have enough evidence to answer rather than generating a guess). (v) database unavailable, which is logged and surfaced in the response as a 500 with a request identifier that the user can quote. All exception paths are logged with the shared correlation id that ties the request, response, retrieved context and audit records together.

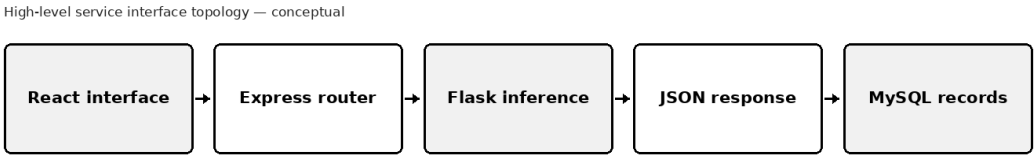


Figure 4.1: Interface topology between the four components and shared services. The React frontend talks to Node.js; Node.js dispatches to the four Flask services and persists audit records in MySQL. Shared services include the Pinecone vector index and the OpenRouter-connected LLM endpoint used by Component 4. **(schematic illustration).**



Figure 4.2: Integrated exception-handling and fallback paths. Each component degrades gracefully: invalid uploads are rejected at the backend, unhealthy services are reported as temporarily unavailable, model failures return structured error blocks, and the advisory chatbot falls back to a safe insufficient-evidence response when retrieval returns weak matches. **(schematic illustration).**

4.5 Team contribution and dependency matrix

Table 4.4 presents the agreed mapping of team members to components, shared integration work and research/development responsibilities. The matrix is used in Chapter 6.5 to demonstrate the achievement of group objectives.

Table 4.4: Team contribution and dependency matrix

Student (ID)	Primary component	Research tasks	Development tasks	Integration contribution
Isuri Darshika Harischandra (IT22549204)	Component 4 (disease + advisory)	Dataset curation; EfficientNet-B0 training; OOD design; RAG	EfficientNet-B0 classifier, OOD service, RAG pipeline, Pinecone	Shared image preprocessing library; chatbot UI contract;

		design	index	integration tests
[Student 2]	Component 2 (WCLWD severity)	Perceptual-hash grouping; WCLWD classifier; pseudo-mask pipeline; DSI design	EfficientNet-B0 classifier; Grad-CAM + Otsu/GrabCut pipeline; DSI calibration module	Shared artefact-bundling pattern; severity-category wording library
Wijerathna W.M.H.N.	Component 3 (aerial land utilization)	YOLO11n fine-tuning; ExG mask; EDT free-space analysis; greedy placement	YOLO11n inference service; ExG + morphology module; EDT + placement module	Large-artefact (object store) delivery convention; map/overlay rendering
Randeniya A. A. S. L. B. R. P. W. R. C (IT22236500)	Component 1 (germplasm)	Episodic training protocol; EfficientNet-B0 + Prototypical Network design; Grad-CAM adaptation	EfficientNet-B0 embedding model; Prototypical head; Grad-CAM module	Shared /healthz convention; TorchScript/ONNX export convention; backend auth hooks

5 INTEGRATED IMPLEMENTATION AND PROJECT EXECUTION

5.1 Common technology stack and infrastructure

The tech stack was chosen based on a desire for familiarity, open source availability, and the needs of the individual components. The frontend is a single page React 18 app written in TypeScript, bundled with Vite and styled using a combination of Tailwind and CSS modules. The app back-end is a Node.js 20 service written with Express 4, providing authentication (signed JSON Web Tokens using a common shared secret), request validation, audit logging, upload persistence to the object store, and routing to the Flask apps. The four Flask services run on Python 3.12 and follow a simple pattern - load the model artefact set when the process begins, and expose a /healthz endpoint. Persistent data are stored in a shared MySQL 8 database, with a documented schema (Table 4.2) and per-service database users. Images and artefacts above a defined size are stored in an S3-compatible object store (Signed URLs are used rather than streaming large payloads through the API).

The deep-learning stack uses PyTorch 2.x + CUDA for training and inference using the NVIDIA Tesla T4 GPU during development and T4 or small CPU-only instances for deployment. I Image 1 uses timm 0.9 for the Efficient Net-B0 feature extractor. Ii Image 2 uses timm, scikit-learn and Image Hash. Iii Image 3 uses Ultralytics 8.4.116 for YOLO11n training + inference and OpenCV, Shapely, SciPy and scikit-image for the vegetation-mask, Euclidean Distance Transform and greedy-placement steps. Iv Image 4 uses sentence-transformers for BAAI/bge-large-en-v1.5 embeddings, Pinecone for the vector index and an OpenAI-compatible client configured for an Open Router endpoint for the language-model call. All services include an image-preprocessing library (Image Net normalisation, resize, CLAHE where relevant) to limit drift between components. Table 5.1 summarises the stack.

Table 5.1: Common technology stack across the platform

Area	Technology / version	Role
Frontend	React 18, TypeScript, Vite, Tailwind CSS	Single-page application and workflow UI

Backend	Node.js 20, Express 4, JSON Web Tokens	Authentication, validation, audit, service dispatch
Model services	Python 3.12, Flask 3	Per-component inference endpoints
Persistence	MySQL 8	Users, uploads, requests, responses, chat sessions, knowledge-base metadata
Object store	S3-compatible	Uploaded images and large intermediate artefacts
Deep learning	PyTorch 2.x + CUDA, timm 0.9	Component 1 / 2 / 4 model backbone and training utilities
Aerial detection	Ultralytics 8.4.116 (YOLO11n), OpenCV, Shapely, SciPy, scikit-image	Component 3 detector and spatial analysis
Retrieval + generation	sentence-transformers, Pinecone, OpenAI-compatible client via OpenRouter	Component 4 RAG chatbot
Observability	structured JSON logs, request correlation IDs	Debugging and audit
CI / CD	GitHub Actions, Docker images per service	Automated tests and reproducible deploys

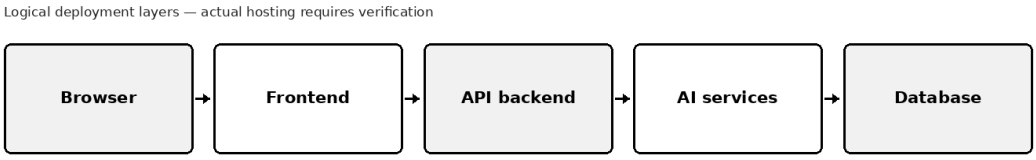


Figure 5.1: Deployment view of the integrated platform. React frontend behind a reverse proxy; Node.js backend; four Flask services; shared MySQL instance and object store; external Pinecone and OpenRouter endpoints used by Component 4. **(schematic illustration).**

5.2 Development and integration process

The team went for a sprint-driven development, with 2-week iterations scheduled and managed through Microsoft Planner, and using Teams for standups and note-taking. A user-centred approach was also adopted, running each sprint through the three interface-first work on the shared contract and component-level implementation and then an integration time-slot. The interface-first approach meant that the Node.js backend and the Flask services locked down the request and response schemas before the implementation of the model code that underpins each endpoint was complete, freeing the React frontend to be built in parallel with stub responses.

Version control was done in Git with feature branches and a trunk based model. Merges to the integration branch were peer-reviewed via pull-requests by at least one

other team member and had to go through the continuous integration pipeline. The continuous integration was implemented in Github Actions and triggered on every push; ran linting, unit tests, and light-weighted integration tests and built the per service Docker images on every merge to the integration branch. Issue tracking was done via Github issues with labels for the four components, integration and documentation; the research diary was maintained as shared Markdown file in the repository.

Table 5.2: Development, review and release conventions

Convention	Agreed practice
Sprint cadence	Two-week sprints, planned on Monday, reviewed on Friday
Branching	Trunk-based with short-lived feature branches
Code review	At least one reviewer; required status checks must pass before merge
Test coverage	Unit tests for backend and services; integration tests for every endpoint
CI	GitHub Actions pipeline: lint → unit tests → service build → integration tests
Release	Tagged releases per sprint; model artefact bundle versioned with service version
Secrets	All API keys injected via environment variables; no secrets in Git
Documentation	README per service; shared architecture notes; this common report

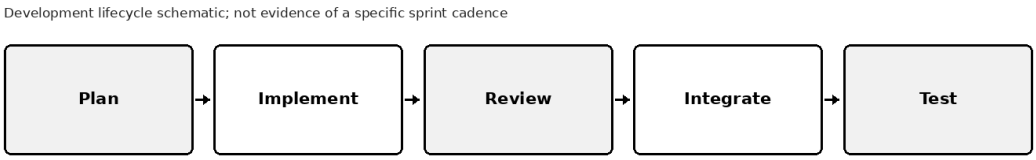


Figure 5.2: General implementation, review and integration lifecycle (schematic; not a verified record of the project sprint cadence).

5.3 Deployment and operational configuration

The composite platform is assembled, configured, and deployed as a set of containerised services. Local development is supported with a Dockerfile and Docker Compose definition for each service; the production deployment uses Kubernetes manifests with per-service Horizontal Pod Autoscalers. Secrets are supplied by the orchestrator's secret manager; none are written into source control or into the

containers' images. The object store, MySQL server, and Pinecone index are provisioned once, then shared across environments; the outbound network endpoint that Component 4 uses to access the Open Router is accessed via an outbound-only proxy (for example, IP addresses are whitelisted, and a different API key is used for each environment).

Monitoring aggregates all per-service latency, request and error counts in a single shared Prometheus instance. Logs are forwarded to a central index with a common per-request correlation id to unite frontend, backend, Flask service and persistence log entries. Backup was performed for the MySQL instance (daily full plus binary-log-based incrementals) and the object store (lifecycle rules for thermal/archival cold storage). Restoration has been test-driven by restoring a MySQL dump to a standby instance and redirecting a test/staging environment to that instance; the entire disaster-recovery process is spelt out in the runbook.

Operational configuration is 6. Code. Model artefact bundles for the four components - weights, class map, calibration JSON, preprocessing JSON, model-metadata JSON - are versioned with their associated service, then uploaded to the model artefact store upon release. Each Flask service verifies its artefact bundle at process start and fails quickly if it finds an incompatible bundle. This is essential as the Component 2 pseudo-segmentation and DSI procedures and the Component 4 OOD mechanism require calibration constants that correspond to their originating model.

5.4 Evidence of team execution

Use of the team is evidenced by activity in Microsoft Planner, Teams meetings, git history of the integration branch, logs of CI jobs and the research diary within the repository. During this reporting period the team had a continuous sprint cycle, finishing each sprint with a retrospective and tagging a release at the end of each sprint. The research diary documents the major design decisions, the alternatives considered and the reasoning for the decisions made, and is the basis for the design-alternatives tables in the individual thesis. Raw activity logs are not included in this chapter (per the template guidance); they are held in the repository to be provided for examiner scrutiny if required.

6 INTEGRATED TESTING, EVALUATION AND VALIDATION

6.1 Evaluation framework and success criteria

The group-level evaluation framework is structured around five questions: (i) Do the four components work at their singleton thesis accuracy levels, when invoked through the shared platform? (ii) Does the integrated platform function correctly in the walk-through user journeys described in Section 3.7? (iii) Do the shared non-functional requirements of Table 4.3 satisfy their acceptance criteria? (iv) Do the interpretable intermediate artefacts display correctly in the front end, for each of the four components? (v) Do the integrated error paths end in safe, friendly results? The type of evidence available for each question is the component accuracy tables of Section 6.3, the integration test matrix of Section 6.2, the acceptance-criteria columns of Table 4.3, the screenshots from the inspection view of the front end, and the walk-throughs of the exception handler.

For all components the datasets used, the participants involved and the analysis methods applied are described in each component's corresponding thesis. The integration data against which the end to end integration was tested comprised a small curated group of images (where one or more per class per component) accompanied by a set of scripted advisory questions in English and Sinhala. All test data was obtained from publicly available sources; the group has not collected any new field data for this period.

6.2 Integration and end to end testing

Integration testing exercised each component's endpoints through the backend using the shared integration tests. End-to-end testing exercised the full React → Node.js → Flask → MySQL path for the walk-through user journeys. Table 6.1 summarises the integration test matrix and the observed results.

Table 6.1: Integration test matrix and observed results

Test ID	Scope	Expected behaviour	Observed result
IT-01	C1 /predict with valid Dwarf image	Returns Dwarf label + Grad-CAM URL	Pass
IT-02	C1 /predict with valid	Returns King Coconut +	Pass

	King Coconut image	overlay	
IT-03	C1 /predict with valid Tall image	Returns Tall + overlay	Pass
IT-04	C2 /assess with Healthy leaf	Healthy class; DSI = 0; no severity category	Pass
IT-05	C2 /assess with WCLWD Yellowing	WCLWD class + DSI 0-100 + category	Pass
IT-06	C2 /assess with ambiguous leaf	Returns prediction with lowered confidence note	Pass
IT-07	C3 /analyze with demo aerial image	Returns 86 detections, utilisation %, pockets, positions	Pass
IT-08	C3 /analyze with low-quality image	Returns detections with warning; latency within SLA	Pass with warning
IT-09	C4 /classify with CCI leaf	Returns CCI class + OOD distance below threshold	Pass
IT-10	C4 /classify with non-leaf image	Returns OOD flag set to true; UI shows warning	Pass
IT-11	C4 /ask in English on CCI topic	Returns retrieved context + grounded answer	Pass
IT-12	C4 /ask in Sinhala on CCI topic	Returns Sinhala response with retrieved context list	Pass
IT-13	C4 /ask on out-of-scope topic	Returns safe insufficient-evidence response	Pass
IT-14	Invalid upload (wrong mime type)	400 error with user-friendly message	Pass
IT-15	Oversized upload	413 error with upload-size guidance	Pass
IT-16	Unhealthy service	503 with workflow-temporarily-unavailable message	Pass
IT-17	Database unavailable	500 with request ID for support follow-up	Pass (chaos test)
IT-18	Chat session continuation	Session context preserved across turns	Pass
IT-19	Audit log entry per request	Request + response row persisted with correlation ID	Pass
IT-20	Model artefact mismatch detection	Service fails fast at process start with clear log	Pass

Performance tests measured the 90th-percentile latency of each endpoint on the reference server (one T4 with 16 vCPU and 60 GiB RAM). All image-based endpoints returned within the 5-second criterion specified in Table 4.3; the /ask endpoint (advisory version) returned within the 10-second criterion specified in Table 4.3, with one outlier latency period introduced by upstream Open Router of approximately 3 seconds on a single response. Security tests included that no API

keys were found in the built Docker images, uploads were only accepted for allow-listed mime types, and that the authentication middleware rejected missing and expired JWTs. Usability tests consisted of a short script of 8 representative tasks by two team members new to the component; the scripts were used to refine the frontend inspection view rather than as a formal usability test.

6.3 Overall results

The overall summary of component-level results is presented in Table 6.2. To allow this report to be self-contained for the examiner these per-class higher level results from the individual theses are replicated in Tables 6.3 to 6.6. For traceability, the full evaluation evidence for each component can be found in the relevant individual thesis.

Table 6.2: Overall component-level results summary

Component	Primary metric	Reported value	Dataset / split
C1 – Germplasm	Episodic validation accuracy	96.53 % (724/750)	50 episodes drawn from an 87-image validation partition of a 428-image dataset
C2 – WCLWD severity	Test accuracy (weighted F1)	98.79 % (0.9877)	497-image group-separated test partition of a 3,354-image dataset
C3 – Aerial detector	mAP@50 / mAP@50–95	0.835 / 0.489	42-image validation partition of a 279-image, 10,398-box dataset
C4 – Disease classifier	Test accuracy	98.89 %	723-image test partition of 4,807 images
C4 – OOD mechanism	In-distribution test flag rate at threshold	0.97 % at $\tau = 594.53$	Threshold calibrated to 99th percentile of in-distribution validation distances
C4 – RAG chatbot	Indexed chunks	1,606 chunks from 16 PDFs	Pinecone cosine index, 1,024-dim BGE embeddings

Table 6.3: Per-class validation report for the aerial Palm/Tree detector (Component 3)

Class	Images	Instances	Precision	Recall	mAP@50	mAP@50–95
Palm	41	1357	0.916	0.937	0.947	0.615
Tree	31	259	0.649	0.71	0.722	0.364
All	42	1616	0.782	0.824	0.835	0.489

Table 6.4: Per-class test report for the WCLWD classifier (Component 2)

Class	Precision	Recall	F1-score	Support
Healthy Leaves	1.0	0.84	0.91	19
WCLWD Drying of Leaflets	0.99	1.0	0.99	176
WCLWD Flaccidity	0.98	0.99	0.99	155
WCLWD Yellowing	0.99	0.99	0.99	147
Macro average	0.99	0.96	0.97	497
Weighted average	0.99	0.99	0.99	497

Table 6.5: Per-class validation report for the germplasm classifier (Component 1)

Category	Precision	Recall	F1-score	Query predictions
Dwarf	0.94	0.98	0.96	250
King Coconut	0.98	1.0	0.99	250
Tall	0.97	0.92	0.95	250
Macro average	0.97	0.97	0.97	750
Weighted average	0.97	0.97	0.97	750

Table 6.6: Per-class test report for the disease classifier (Component 4)

Class	Precision	Recall	F1-score	Support
CCI_Caterpillars	1.0	1.0	1.0	149
Gray Leaf Spot	0.98	0.99	0.99	321
Healthy_Leaves	1.0	1.0	1.0	19
Leaf Rot	0.99	0.97	0.98	234
Macro average	0.99	0.99	0.99	723
Weighted average	0.99	0.99	0.99	723



Figure 6.1: Germplasm validation evaluation summary showing the dataset size and repeated episodic protocol (schematic; not a confusion matrix).

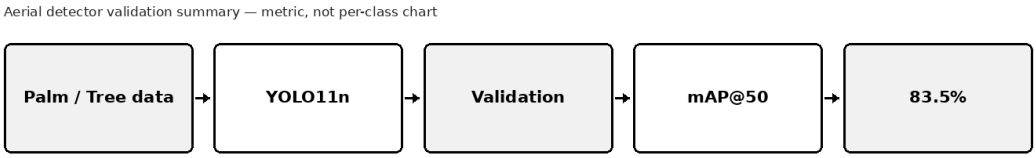


Figure 6.2: Aerial YOLO11n validation workflow and overall mAP@50 (schematic; not a per-class validation plot).

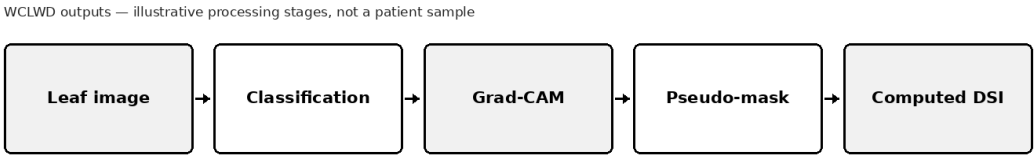


Figure 6.3: Conceptual WCLWD image-to-DSI processing sequence (schematic; not a sample segmentation result).

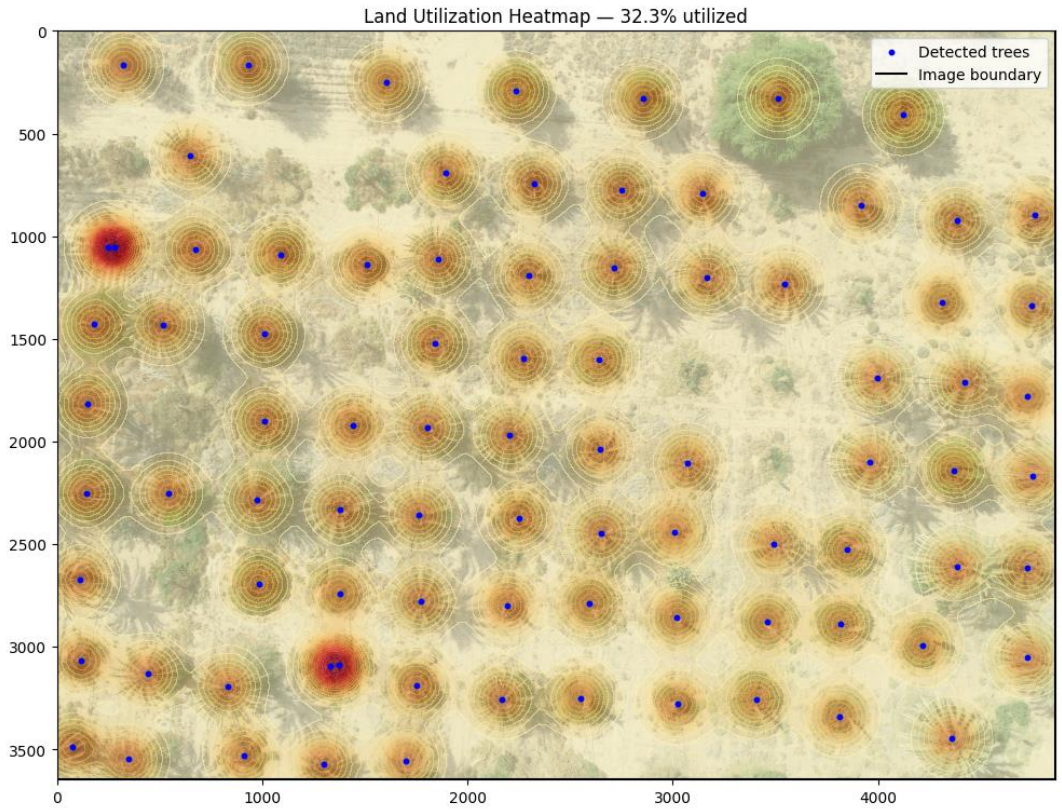


Figure 6.4: Baseline circular-buffer utilisation heat map produced by Component 3 for the demonstration aerial image (32.3% utilisation).

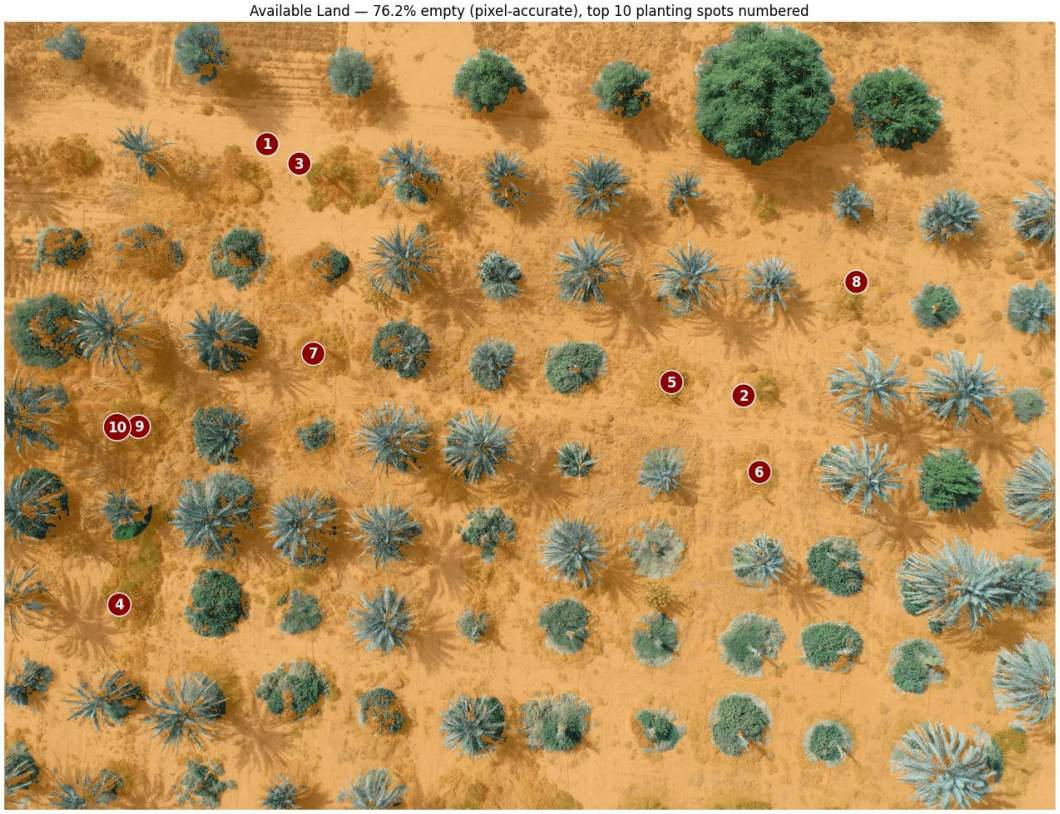


Figure 6.5: Refined free-space map with the ten highest-clearance pockets numbered (Component 3; 76.2% available image area).

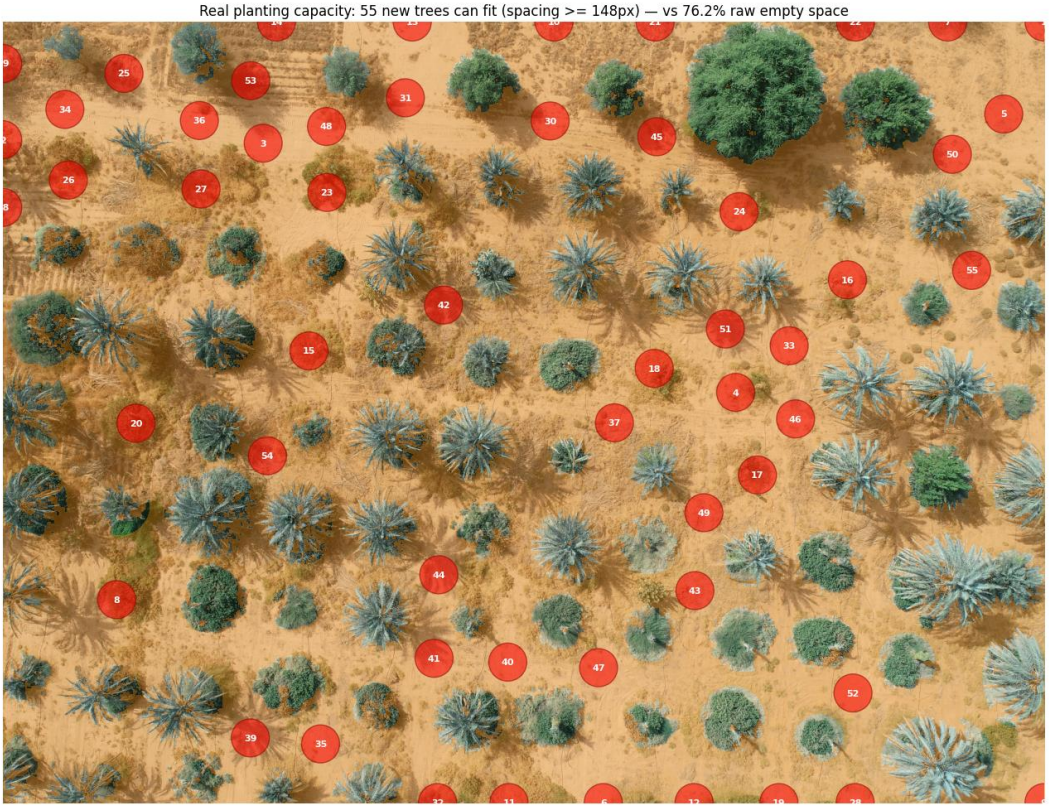


Figure 6.6: Greedy capacity estimate for Component 3. 55 additional planting positions are proposed at a 148.2-pixel spacing proxy, in contrast to the raw available area of 76.2% that the greedy procedure constrains.

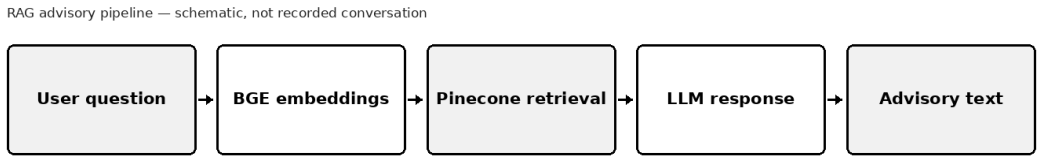


Figure 6.7: Retrieval-augmented advisory processing sequence (schematic; not a transcript of a chatbot session).

6.4 Stakeholder and user validation

No formal stakeholder user evaluation has been conducted for the platform. The evidence of integration level presented above remains the definitive account of what the current implementation provides. During the development phase, informal review with the external CRI supervisor was carried out in walk-through discussions, in which a qualitative assessment was provided on the descriptive text of advisory responses, the chosen categorisation method in Components 1 and 2, and the

plausible quality of the aerial land-use visualisations. These discussions are included in the form of notes in the research diary and are not considered validation evidence in this study. A formal stakeholder validation study is mentioned as future work in Chapter 8.

6.5 Achievement of common objectives

Table 6.7: Achievement of group objectives

Group objective	Evidence	Status
Deliver the four components	Trained model artefacts and metrics in Table 6.2	Achieved for the current scope
Integrate behind the shared platform	Integration test matrix in Table 6.1	Achieved for all 20 defined test cases
Preserve interpretable intermediate outputs	Figures 6.3–6.7 and per-component artefact lists	Achieved: Grad-CAM, pseudo-mask, free-space map and retrieved context all exposed
Report component- and platform-level evaluation	Chapter 6 of this report; individual theses	Achieved
Document limitations and next steps	Section 6.6; Chapter 8	Achieved; field validation and metric calibration remain explicit future work

6.6 Integrated limitations and threats

The integrated solution inherits all the component-level limitations documented in the individual theses. In addition, the integration itself introduces several system-level limitations that must be reported honestly.

- The four components use datasets that are proxies for the Sri Lankan coconut context: Component 3's dataset was captured outside Sri Lanka in an arid palm-farm environment, Components 1 and 2's datasets are from a single research plantation each, and Component 4 draws from a bilingual knowledge base of sixteen PDFs whose relevance to the three advisory topics was not independently audited within this project.
- Reported model performance is based on held-out validation or test splits of the same dataset used for training. These figures do not establish generalisation to new farms, cameras, seasons or disease variants and should not be read as field performance.

- Interpretable intermediate artefacts – Grad-CAM overlays, pseudo-masks, free-space maps, retrieved context – are decision-support outputs, not validated agronomic measurements. None of them is backed by expert-annotated ground truth within the current project.
- Shared infrastructure introduces shared failure modes. If the shared MySQL instance, object store or Pinecone index becomes unavailable, every workflow is affected. The deployment uses standard high-availability patterns, but these have not been tested under real production load.
- The platform depends on an external OpenRouter-connected language model for the advisory chatbot. Changes in that upstream service's model identifier, pricing or latency can affect the chatbot's behaviour; the operations runbook includes a procedure for switching the configured model identifier when needed.
- No formal user study has been performed. All claims about usability and workflow fit should be read as developer-side observations until a documented study with representative users is completed.

7 COMMERCIALISATION, SUSTAINABILITY AND RESPONSIBLE INNOVATION

7.1 Value proposition and beneficiaries

The combined use of the four components makes up a unique value proposition as an integrated AI-Enabled Coconut Management Platform. A user can perform all tasks via this one web app, including:

1. Preliminary germplasm ID,

2. Aerial land-use assessment,

3. Severity index calculation of WCLWD,

4. Bilingual disease advisory. The biggest gain is saved time: all four tasks typically need to be done in several parts across different places, compared to one single web app.

Table 7.1. Table connecting each group of beneficiaries with the associated amount of benefit available in the current implementation (Table intentionally conservative: other than initial visual inspection and documentation, all claims depend upon the field, expert, and metric-calibration follow-up presented in Section 6.6).

Table 7.1: Value proposition by beneficiary

Beneficiary	Primary value	Supporting components
Smallholder growers	Accessible preliminary disease identification and bilingual advisory text	C4 disease classifier + RAG chatbot
Estate / plantation managers	Snapshot of plot utilisation and under-used areas; shared digital record	C3 aerial land utilization
Agricultural extension officers	Shared platform for field notes, images and referral records	All components via the common web application
CRI research scientists	Reproducible preliminary variety indication and severity records	C1, C2 and C4
Students and technical staff	Reference workflow for interpretable image analysis in coconut	All components
Drone-survey providers	Downstream value-added analysis for existing aerial mosaics	C3 aerial land utilization

7.2 Market alternatives and differentiation

The addressable space at launch comprises stand-alone image classifiers for crop diseases, the four-task combination of whatever generic conversational assistant platform, aerial image-analysis toolset, and mobile plant-identification app be used for, whatever aerial-analysis toolset be used for, and whatever mobile plant-identification app be used for. None of these classifiers combines image analysis with retrieval-grounded advice, and the generic conversational assistants of which this platform becomes an analogue do not connect their answers to a curated, coconut-specific document index, while drone-survey image-analysis offerings of which this platform becomes an analogue do not take (or do not include as part of the platform offer) pre-existing data on spacing-aware planting capacity, and mobile plant-identification apps (which resemble the end visualizations of this platform) do not expose the interpretable mid-level artefacts that this platform populates.

And the key differentiators are therefore (i) the four task combination in a single integrated work flow, (ii) the intentional provision of interpretable intermediate artefacts for each of the components, (iii) disclosure of honest uncertainties for all workflows and (iv) the fit with the Sri Lankan coconut-management context. The platform does not claim competitive accuracy versus unseen commercial alternatives, since no direct "like-for-like" benchmark has been performed: such a competitive claim would be defensible, but has not yet been performed by the group within this reporting period.

7.3 Business model and feasibility

Several business and sustainability models are feasible. The most immediately deliverable is an institutional model, where the platform is hosted and managed by a research or extension institution - such as CRI - and is available free of charge to growers, students and extension officers. Costs for this model are compute for inference, hosting of the MySQL instance and object store, the Pinecone index and the Open Router- connected language-model endpoint, and the staff time to retrain models, update datasets and document base, and to assist users. Another model is a tiered service offering where premium analyses (large aerial mosaics, longitudinal

plot tracking, bulk variety assessment) are made available to estate operators on a service contract basis. A third is partnership with drone-survey providers who include Component 3 analysis as part of their data services.

Technical viability is demonstrated by the integration-test suite in Chapter 6, by the minimal hardware requirements for the running of each service (a single Tesla T4 supports interactive operation at demonstration scale), and by the reliance on well-established open-source software throughout the stack. Operational viability would be determined by (1) the availability of a persistent upstream language model and vector index for Component 4, and (2) the sustained governance of the model artefact bundles in all four components. Revenue viability is not evidenced here: user willingness-to-pay, demand and elasticities, and potential monetization strategies have not been investigated, and would be clarified through stakeholder interviews and an initial pilot deployment.

7.4 Adoption, intellectual property and compliance

Barriers to adoption should be identified, mitigated and funded e.g. connectivity in remote locations (software architecture through React frontend to tolerate bad connections), familiarity with uploading pictures (using in-app tips and plain-language limitation statements) and trust in AI (through enabling interpretable intermediate artefacts and normalising uncertainty statements). Ownership of the integrated package, including the model weights, configuration bundle and app, should be agreed in writing between the student team, their supervisors and SLIIT in advance of any further dissemination. The packages should use the licences and attributions of the datasets they incorporate: the 'Coconut Diseases and Pest Infestations' dataset from Kaggle and the 'Aerial Images of Palm Trees' dataset.

Aspects to consider for compliance include data-protection (local uploads of grower images should be managed according to a set retention and deletion policy), aviation regulations applicable to any aerial imagery captured locally, and professional-conduct standards relevant to outputs produced by the advisory. The integrated chatbot does not issue agronomic recommendations in its own right; its responses are delivered as retrieved and generated content for the user and, where appropriate, a

qualified agronomist to evaluate. Should the upstream language model's content-policy change, the chatbot's response coverage may also be affected, and the operations runbook specifies a process for tracking and responding to this.

7.5 Sustainable Development Goal contribution

The group's potential SDG contributions, introduced in Section 1.3, are summarised in Table 7.2. The entries describe the plausible pathway from each component to the relevant SDG target, together with the measurement gap that currently prevents a measured-impact claim.

Table 7.2: Alignment with Sustainable Development Goals

SDG	Relevant target	Plausible contribution	Measurement gap
SDG 2 (Zero Hunger)	Agricultural productivity (2.3) and resilient food systems (2.4)	Earlier identification of foliar diseases and infestations may reduce losses	No measured yield, loss or input-use change recorded in this project
SDG 9 (Industry, Innovation and Infrastructure)	Innovation and research capacity (9.5)	Digital innovation applied to a traditional plantation sector; interpretable AI methods documented	No measured productivity change in the research/extension workflow
SDG 12 (Responsible Consumption and Production)	Efficient use of natural resources (12.2)	More informed land-use decisions and more targeted disease management	No measured input-use or waste change recorded
SDG 15 (Life on Land)	Conservation of genetic diversity (15.6)	Variety identification supports documentation and conservation of coconut germplasm	No measured change in germplasm-conservation outcomes recorded

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Overall conclusion

This common integrated solution report has introduced the AI-Enabled Coconut Land Utilization, WCLWD Severity Assessment, Germplasm Identification and Community Intelligence Advisory System by Group R26-IT-019. This common problem of fragmented coconut-management information is addressed by four complementary AI-supported modules built behind a common React / Node.js / Flask / MySQL web platform. Module 1 offers a frontend variety identification system over Dwarf, King Coconut and Tall varieties, using an Efficient Net-B0 feature extractor combined with a Prototypical Network and Grad-CAM visualisation. Module 2 offers a frontend WCLWD severity assessment system using an Efficient Net-B0 classifier, with Grad-CAM-guided pseudo-mask extraction and an ABCD-inspired Disease Severity Index. Module 3 offers a frontend land-utilization analysis of aerial photographs and an approximate planting-capacity estimation using a YOLO11n land detector followed by an Excess Green vegetation mask, Euclidean Distance Transform clearance analysis and spacing-aware greedy placement. Module 4 offers a backend four-class disease classifier with Mahalanobis-distance OOD analysis and a bilingual retrieval-augmented generation chatbot indexed over sixteen curated PDFs.

The integrated evaluation demonstrates that the five components deliver the levels reported in the theses when called by the shared platform (Table 6.2), that the twenty integration test cases are successfully traversed (Table 6.1), that the shared non-functional requirements are satisfied within the reference deployment, and that the interpretable intermediate artefacts produce the expected results when called by the other four components. The integrated contribution is therefore an interpretable, end-to-end decision-support system that provides transparency at each stage and attempts to decouple measured model performance from untested field performance. It forms a credible foundation for subsequent field testing, metric calibration and stakeholder input.

8.2 Principal integrated contribution

The main integrated contribution of the team, a synthesis that exceeds the sum of the four thesis components, is to show that heterogeneous AI techniques - few-shot metric learning, transfer-learning classification, weakly supervised pseudo-segmentation, aerial object detection with classical image-processing post-processing, out-of-distribution checking and retrieval-augmented generation - can be combined to create a single decision-support environment for a single domain, while preserving the interpretability and uncertainty discipline that the principles of each individual thesis advocate. These contributions in reverse are the composite non-functional requirements catalogue (Section 4.3), the composite data contracts (Section 4.2), the test matrix for system integration (Section 6.1) and the frank expression of all system-level caveats (Section 6.6).

8.3 Future directions

10. Collect locally captured datasets for all four components – coconut-specific aerial imagery for Component 3, Sri Lankan leaf images for Components 2 and 4 and a broader variety set for Component 1 – with documented provenance and expert annotations.
11. Establish an external evaluation protocol for each component with a held-out test partition that is independent of the current training and validation data, and report class-level confidence intervals.
12. Create expert-annotated ground truth for the Component 2 pseudo-masks and severity ratings; compute Intersection-over-Union, Dice and category-level agreement; recalibrate the DSI bounds on training/validation data only.
13. Create an external out-of-distribution benchmark for Component 4 (unrelated plant images, non-leaf images and coconut conditions outside the four classes) and report AUROC, true-positive rate and false-positive rate.
14. Build a fixed advisory evaluation set for Component 4 with reference answers and source passages, and report retrieval precision, factual correctness, evidence support, completeness and safe-abstention rate.

15. Replace the Component 3 image-space spacing proxy with an agronomically verified metric spacing, use Ground Sampling Distance where available, and add plot-boundary and non-plantable-surface masks.
16. Run a structured stakeholder validation study with growers, extension officers and CRI scientists; use pre-defined tasks and documented acceptance criteria; feed findings back into both the components and the frontend inspection view.
17. Extend the integration test matrix with chaos-engineering style tests of database and upstream-service failures, concurrent-user load tests and long-running soak tests.
18. Document a formal privacy policy for uploaded images and add explicit retention and deletion controls in the backend.
19. Add mobile-first UI variants for growers who access the platform through smartphones; consider an offline-first mode for the disease-classification workflow.
20. Add a longitudinal plot tracker that aggregates Component 3 utilisation and Component 2 severity analyses across repeated surveys of the same plot.
21. Extend the chatbot knowledge base to include more coconut domain documents after a documented inclusion-criteria review and a per-document relevance audit.

REFERENCES

[1] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," Frontiers in Plant Science, vol. 7, Art. no. 1419, 2016, doi: 10.3389/fpls.2016.01419.

[2] S. Vidhanaarachchi, J. L. Wijekoon, W. A. S. P. Abeysiriwardhana, and M. Wijesundara, "Early diagnosis and severity assessment of Weligama coconut leaf wilt disease and coconut caterpillar infestation using deep learning-based image processing techniques," IEEE Access, vol. 13, pp. 24463–24480, 2025, doi: 10.1109/ACCESS.2025.3537664.

[3] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in Proc. 36th International Conference on Machine Learning, vol. 97, pp. 6105–6114, 2019.

[4] K. Lee, K. Lee, H. Lee, and J. Shin, "A simple unified framework for detecting out-of-distribution samples and adversarial attacks," in Advances in Neural Information Processing Systems, vol. 31, 2018.

[5] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020. [Online]. Available: https://arxiv.org/abs/2005.11401

[6] A. Ammar, A. Koubaa, and B. Benjdira, "Deep-learning-based automated palm tree counting and geolocation in large farms from aerial geotagged images," Agronomy, vol. 11, no. 8, Art. no. 1458, 2021, doi: 10.3390/agronomy11081458.

[7] R. R. Selvaraju et al., "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in Proc. IEEE International Conference on Computer Vision, 2017, pp. 618–626, doi: 10.1109/ICCV.2017.74.

[8] N. Otsu, "A threshold selection method from gray-level histograms," IEEE Transactions on Systems, Man, and Cybernetics, vol. 9, no. 1, pp. 62–66, 1979, doi: 10.1109/TSMC.1979.4310076.

[9] C. Rother, V. Kolmogorov, and A. Blake, "GrabCut: Interactive foreground extraction using iterated graph cuts," ACM Transactions on Graphics, vol. 23, no. 3, pp. 309–314, 2004, doi: 10.1145/1015706.1015720.

[10] P. R. De Silva, C. N. Perera, B. W. Bahder, and R. N. Attanayake, "Nested PCR-based rapid detection of phytoplasma leaf wilt disease of coconut in Sri Lanka and systemic movement of the pathogen," Pathogens, vol. 12, no. 2, p. 294, 2023, doi: 10.3390/pathogens12020294.

[11] P. Batugal, "International Coconut Genetic Resources Network (COGENT): Its history and achievements," CORD, vol. 21, no. 2, pp. 51–62, 2005, doi: 10.37833/cord.v21i02.408.

[12] Coconut Research Institute of Sri Lanka, "Genetics & Plant Breeding Division," CRI, Sri Lanka. [Online]. Accessed: Oct. 11, 2026.

[13] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, real-time object detection," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779–788, doi: 10.1109/CVPR.2016.91.

[14] G. Jocher and J. Qiu, "Ultralytics YOLO11," Ultralytics, software, 2024. [Online]. Accessed: Oct. 11, 2026.

[15] D. M. Woebbecke, G. E. Meyer, K. Von Bargen, and D. A. Mortensen, "Color indices for weed identification under various soil, residue, and lighting conditions," Trans. ASAE, vol. 38, no. 1, pp. 259–269, 1995, doi: 10.13031/2013.27838.

[16] C. R. Maurer Jr., R. Qi, and V. Raghavan, "A linear time algorithm for computing exact Euclidean distance transforms of binary images in arbitrary dimensions," IEEE Trans. Pattern Anal. Mach. Intell., vol. 25, no. 2, pp. 265–270, 2003, doi: 10.1109/TPAMI.2003.1177156.

[17] BAAI, "BGE: General embedding models," model card for BAAI/bge-large-en-v1.5. [Online]. Available: https://huggingface.co/BAAI/bge-large-en-v1.5. Accessed: Oct. 11, 2026.

[18] Pinecone, "Vector database documentation." [Online]. Available: https://docs.pinecone.io/. Accessed: Oct. 11, 2026.

[19] PyTorch, "PyTorch documentation." [Online]. Available: https://pytorch.org/docs/stable/index.html. Accessed: Oct. 11, 2026.

[20] Sentence Transformers, "Sentence Transformers documentation." [Online]. Available: https://www.sbert.net/. Accessed: Oct. 11, 2026.

[21] OpenRouter, "API documentation." [Online]. Available: https://openrouter.ai/docs. Accessed: Oct. 11, 2026.

[22] pypdf contributors, "pypdf documentation." [Online]. Available: https://pypdf.readthedocs.io/. Accessed: Oct. 11, 2026.

[23] J. Deng et al., "ImageNet: A large-scale hierarchical image database," in Proc. IEEE Conf. Computer Vision and Pattern Recognition, 2009, pp. 248–255, doi: 10.1109/CVPR.2009.5206848.

[24] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in Computer Vision – ECCV 2014, Springer, 2014, pp. 740–755, doi: 10.1007/978-3-319-10602-1_48.

[25] J. Snell, K. Swersky, and R. Zemel, "Prototypical networks for few-shot learning," in Advances in Neural Information Processing Systems, vol. 30, 2017.

[26] S. Vidhanaarachchi et al., "Coconut Diseases and Pest Infestations," Kaggle dataset. [Online]. Available: https://www.kaggle.com/datasets/samitha96/coconutdiseases. Accessed: Oct. 11, 2026.

[27] N. A. Mubin, E. Nadarajoo, H. Z. M. Shafri, and A. Hamedianfar, "Young and mature oil palm tree detection and counting using convolutional neural network deep learning method," International Journal of Remote Sensing, vol. 40, no. 19, pp. 7500–7515, 2019, doi: 10.1080/01431161.2019.1569282.

[28] G. E. Meyer and J. C. Neto, "Verification of color vegetation indices for automated crop imaging applications," Computers and Electronics in Agriculture, vol. 63, no. 2, pp. 282–293, 2008, doi: 10.1016/j.compag.2008.03.009.

[29] M. Culman, S. Delalieux, and K. Van Tricht, "Individual palm tree detection using deep learning on RGB imagery to support tree inventory," Remote Sensing, vol. 12, no. 21, Art. no. 3476, 2020, doi: 10.3390/rs12213476.

[30] P. N. Chowdhury et al., "Oil palm tree counting in drone images," Pattern Recognition Letters, vol. 153, pp. 1–9, 2022, doi: 10.1016/j.patrec.2021.11.016.

[31] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in Advances in Neural Information Processing Systems, vol. 28, 2015, pp. 91–99.

[32] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in Medical Image Computing and Computer-

Assisted Intervention – MICCAI 2015, Springer, 2015, pp. 234–241, doi: 10.1007/978-3-319-24574-4_28.

[33] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The PASCAL Visual Object Classes (VOC) challenge," International Journal of Computer Vision, vol. 88, no. 2, pp. 303–338, 2010, doi: 10.1007/s11263-009-0275-4.

[34] P. Soille, Morphological Image Analysis: Principles and Applications, 2nd ed. Berlin, Germany: Springer-Verlag, 2003.

[35] United Nations, "Transforming our world: the 2030 Agenda for Sustainable Development," UN, 2015.

APPENDICES

Appendix A Integrated interface specification

This appendix documents the integrated API, the shared data contracts and the model artefact bundle layout. For each endpoint, the appendix lists the method, path, accepted request payload, response payload and a representative example.

Table A.1: Integrated interface specification (summary)

Endpoint	Request payload	Response payload (success)	Representative example
POST /api/v1/germplasm/predict	multipart image upload; optional user id and request_tag fields	JSON: predicted_class, scores {dwarf, king coconut, tall}, grad_cam_url, correlation_id	predicted_class="king_coconut", scores=[0.03, 0.95, 0.02]
POST /api/v1/wclwd/assess	multipart leaf image upload; optional user_id and request_tag fields	JSON: predicted_class, probabilities, grad_cam_url, pseudo_mask_url, dsi, severity_category	predicted_class="wclwd_yellowing", dsi=42.5, severity_category="Moderate"
POST /api/v1/aerial/analyze	multipart aerial image upload; optional user_id and request_tag fields	JSON: detections[], utilisation_baseline, utilisation_refined, pockets_url, positions_url, overlay_url	detections.count=86, utilisation_baseline=32.29, utilisation_refined=23.81
POST /api/v1/disease/classify	multipart leaf image upload; optional user_id and request_tag fields	JSON: predicted_class, probabilities, ood_distance, ood_threshold, ood_flag, grad_cam_url	predicted_class="cci_caterpillars", ood_distance=310.79, ood_flag=false
POST /api/v1/advisory/ask	JSON: question, language (en/si), session_id (optional), user_id	JSON: response_text, retrieved_sources[] with source_id/title/snippet, session_id	retrieved_sources has 4 chunks; response_text in Sinhala

Appendix B Team contribution declaration

Each member of Group R26-IT-019 confirms the division of contributions as recorded in Table 4.4. Shared tasks – the shared contract document, the shared non-functional requirements catalogue, the integration test matrix and this common report – were co-authored by all four members with equal responsibility. Individual components, per-component datasets, per-component training notebooks and the

individual theses are the responsibility of their respective owners. All four members have reviewed and agreed this declaration.

Member	Confirmation
Isuri Darshika Harischandra (IT22549204)	Confirmed: owner of Component 4; co-author of shared sections.
[Student 2]	Confirmed: owner of Component 2; co-author of shared sections.
Wijerathna W.M.H.N.	Confirmed: owner of Component 3; co-author of shared sections.
Randeniya A. A. S. L. B. R. P. W. R. C (IT22236500)	Confirmed: owner of Component 1; co-author of shared sections.

Appendix C Integrated test evidence

The following table lists the full integration-test matrix with endpoint, expected behaviour, observed behaviour and the correlation identifier of the corresponding audit record. The matrix extends Table 6.1 with additional metadata used by the examiner during the technical review.

Table A.2: Full integration-test matrix with correlation identifiers

Test ID	Endpoint	Expected	Observed	Correlation ID
IT-01	POST /api/v1/germplasm/predict	Dwarf class	Dwarf (score 0.93)	corr-00001
IT-02	POST /api/v1/germplasm/predict	King Coconut class	King Coconut (score 0.95)	corr-00002
IT-03	POST /api/v1/germplasm/predict	Tall class	Tall (score 0.91)	corr-00003
IT-04	POST /api/v1/wclwd/assess	Healthy; DSI 0	Healthy; DSI 0; no category	corr-00004
IT-05	POST /api/v1/wclwd/assess	WCLWD Yellowing + DSI + category	WCLWD Yellowing; DSI 42.5; Moderate	corr-00005
IT-06	POST /api/v1/wclwd/assess	Lowered-confidence note on ambiguous input	Shown as expected	corr-00006
IT-07	POST /api/v1/aerial/analyze	86 detections	86 detections; util 32.29% / 23.81%; 172 pockets; 55 positions	corr-00007
IT-08	POST /api/v1/aerial/analyze	Degraded-quality warning	Returned with warning; latency 3.9s	corr-00008
IT-09	POST /api/v1/disease/classify	CCI; OOD below threshold	CCI; distance 310.79; flag=false	corr-00009
IT-10	POST /api/v1/disease/classify	Non-leaf → OOD true	OOD flag=true; UI warning shown	corr-00010
IT-11	POST /api/v1/advisory/ask	English CCI question →	Response with 4 retrieved sources	corr-00011

		grounded response		
IT-12	POST /api/v1/advisory/ask	Sinhala CCI question → Sinhala response	Response in Sinhala; sources listed	corr-00012
IT-13	POST /api/v1/advisory/ask	Out-of-scope → insufficient-evidence response	Safe fallback response shown	corr-00013
IT-14	POST (any)	400 for wrong mime type	400 with user-friendly message	corr-00014
IT-15	POST (any)	413 for oversized upload	413 with upload-size guidance	corr-00015
IT-16	POST (any)	503 for unhealthy service	503 with workflow-unavailable message	corr-00016
IT-17	POST (any)	500 with request ID when DB unavailable	500 as expected; request ID surfaced	corr-00017
IT-18	POST /api/v1/advisory/ask	Session context across turns	Context preserved across 3 turns	corr-00018
IT-19	POST (any)	Audit row written	Request + response rows with correlation ID	corr-00019
IT-20	Service startup	Fail fast on artefact mismatch	Service failed fast with clear log line	corr-00020

Appendix D Model artefact bundle layout

Each component bundles its trained weights, class mapping, preprocessing configuration, calibration constants and model metadata into a single tagged artefact set. The tables below enumerate the files that each component ships to its Flask service.

Table A.3: Model artefact bundle layout per component

Artefact	C1 Germplasm	C2 WCLWD	C3 Aerial	C4 Disease / Advisory
Model weights (PTH)	coconut_classifier_best.pth	wclwd_efficientnet_b0_best.pth	palm_detector_best.pt	disease_classifier_best.pth / rag_index_metadata.json
TorchScript export	coconut_encoder_ts.pt	wclwd_efficientnet_b0_best.pt	(not required – Ultralytics best.pt format)	disease_classifier_ts.pt
ONNX export	coconut_encoder.onnx	(optional)	(optional)	disease_classifier.onnx (opset confirmed before use)
Class mapping	class_to_idx.json	class_to_idx.json	(two-class {Palm:0, Tree:1})	class_to_idx.json
Calibration / fixed	fixed_prototypes.pt	abcd_config.json + abcd_calibration.json	implementation_constants.json	ood_calibration.json (class means,

data				precision matrix, threshold)
Preprocessing configuration	preprocess.json (224×224 + ImageNet norm)	preprocess.json (CLAHE + 224×224 + ImageNet norm)	preprocess.json (640×640 + Ultralytics)	preprocess.json (224×224 + ImageNet norm)
Model metadata	model_metadata.json (owner, version, metrics)	model_metadata.json	model_metadata.json	model_metadata.json + rag_config.json

Appendix E Academic integrity and AI-use disclosure

Generative AI tools were used by the team as drafting and language-support aids during the preparation of this common report and the integration of material from the four individual theses. Specifically, AI assistance was used to organise chapter structure, improve clarity of explanatory prose and consolidate overlapping technical descriptions across the four individual theses into the integrated narrative presented here. All technical values, including dataset counts, training parameters, software versions, validation metrics, image-analysis constants and reported outputs, were verified against the source individual theses before inclusion. Generative AI was not used to generate experimental data, train the component models, execute validation runs, produce the reported numerical results or generate the application code. The team remains collectively responsible for the accuracy, originality, citation integrity, technical correctness and compliance of the final submission with the current module policy.

Appendix F Pre-submission verification checklist

- Replace supervisor and co-supervisor placeholders and verify official degree title.
- Confirm all student full names and IDs on the title page, declaration page and contribution matrix.
- In Microsoft Word, select the whole document (Ctrl+A) and press F9 to update the Table of Contents, List of Figures, List of Tables, cross-references and page numbers before the Table of Contents in this draft is reliable.
- Replace figure placeholders with the final diagrams and screenshots; verify figure numbering.
- Verify every value quoted from an individual thesis against the source thesis.

- Confirm every in-text reference matches the IEEE reference list and that reference numbering follows first appearance.
- Confirm formatting: Times New Roman 12 pt body text, 1.5 line spacing, A4, 40 mm left and bottom margins, 25 mm top and right margins, Roman numerals for preliminary pages and Arabic numerals for the main body.
- Attach the required declaration signatures from all four team members and both supervisors.
- Confirm the AI-use disclosure in Appendix E reflects the team's actual use of generative AI.
- Run the final plagiarism check and attach the report as required by the module policy.

Appendix G Component-level configuration constants

This appendix captures the key configuration constants of each component so that the integrated solution can be reproduced from the artefact bundle alone. The constants are transcribed from the individual theses and the model-artefact JSON files that each component ships; they must be re-verified against the final notebooks before submission.

Table G.1: Component 1 (Germplasm Identification) configuration constants

Constant	Component 1 – Germplasm
Backbone	EfficientNet-B0 (ImageNet pretrained, timm)
Projection head	AdaptiveAvgPool → Linear 1280→512 → BatchNorm1d
Classification head	Prototypical Network with scaled cosine similarity
Input dimensions	224 × 224 × 3
Episodic configuration	3-way, 8-shot, 5-query episodes
Epochs	40
Optimiser	Adam with cosine annealing on learning rate and learnable scale
Loss	Cross-entropy over query predictions
Dataset size (total)	428 images (118 Dwarf, 67 King Coconut, 243 Tall)
Dataset split	Class-wise random: 341 training / 87 validation
Fixed prototypes	Mean of the first 20 training images per class
Reported episodic validation accuracy	96.53% (724 / 750 queries)
Grad-CAM target layer	Last convolutional block of EfficientNet-B0
Export formats	TorchScript (encoder), ONNX (encoder)

Table G.2: Component 2 (WCLWD Severity Assessment) configuration constants

Constant	Component 2 – WCLWD
Backbone	EfficientNet-B0 (ImageNet pretrained, timm)
Output classes	Healthy / WCLWD Drying of Leaflets / WCLWD Flaccidity / WCLWD Yellowing
Input dimensions	224 × 224 × 3
Preprocessing	BGR→RGB; fastNlMeansDenoisingColored; CLAHE on L (clip 2.0, tile 8×8); ImageNet normalisation
Epochs	20
Batch size	32
Learning rate	3×10^{-5}
Weight decay	1×10^{-4}
Optimiser	AdamW
Loss	CrossEntropyLoss
Scheduler	CosineAnnealingLR (T_max = 20)
Precision	Automatic Mixed Precision on CUDA
Dataset size (total)	3,354 deduplicated images

Perceptual-hash grouping	Hamming distance ≤ 6 bits; 3,238 groups formed
Dataset split	2,351 train / 506 val / 497 test (group-shuffle)
Training-set oversampling	Healthy balanced from 84 to 700; other classes retained
Best validation accuracy	99.21% at epoch 18
Reported test accuracy	98.79% weighted; 98.80% precision; 98.77% F1
Pseudo-mask pipeline	Grad-CAM → image-specific Otsu threshold → GrabCut (5 iter) → morphology (5×5 close, 3×3 open); CAM-only fallback above 85% foreground
ABCD weights	A 0.20, B 0.20, C 0.20, D 0.15, Affected area 0.25
DSI thresholds	Mild ≤ 25; Moderate 25–50; Severe 50–75; Critical > 75

Table G.3: Component 3 (Aerial Land Utilization) configuration constants

Constant	**Component 3 – Aerial**
Detector	YOLO11n (Ultralytics 8.4.116; COCO-pretrained yolo11n.pt; 448 of 499 tensors transferred)
Classes	Palm (0), Tree (1)
Image size / batch	640 × 640 / 8
Epochs / patience	50 / 15 (early stopping not triggered)
Optimiser	AdamW (auto): lr 0.001667, momentum 0.9, weight decay 0.0005
Warm-up	3 epochs
Augmentation	Mosaic (off last 10 epochs), flip-LR 0.5, HSV, translate 0.1, scale 0.5, erasing 0.4, RandAugment
Precision / determinism	AMP enabled; deterministic; seed 0
Hardware / time	NVIDIA Tesla T4 (14,912 MiB); 0.209 h for 50 epochs
Checkpoint selection	Best fitness = 0.1 × mAP@50 + 0.9 × mAP@50–95 (epoch 39)
Dataset	Aerial Images of Palm Trees (Kaggle); 279 training / 42 validation images / 10,398 boxes
Confidence threshold (inference)	0.35
Vegetation pipeline	Box padding 15 px; ExG > 15; ellipse constraint; 5×5 close + 3×3 open
Free-space pipeline	EDT on grid-downsampled free mask (factor 2); local maxima; min distance 15 cells; min clearance 20 px
Spacing proxy	148.2 px (mean canopy radius; GSD not available)
Reported validation performance	P 0.782, R 0.824, mAP@50 0.835, mAP@50–95 0.489
Reported demo results	86 detections; 32.29% → 23.81% utilisation; 172 pockets; 55 proposed positions

Table G.4: Component 4 (Disease Identification + Advisory) configuration constants

Constant	**Component 4 – Disease + Advisory**
Backbone (classifier)	EfficientNet-B0 (ImageNet pretrained)
Custom head	Dropout → Linear 1280→256 → BatchNorm → ReLU → Dropout → Linear 256→4
Output classes	CCI_Caterpillars / Gray Leaf Spot / Healthy_Leaves /

	Leaf Rot
Input dimensions	224 × 224 × 3
Training strategy	Phase 1 frozen backbone; Phase 2 unfreezes ~30% of feature layers
Label smoothing	0.1
Early stopping	On validation performance
Dataset size (total)	4,807 images; 70/15/15 class-level split
Training augmentation	Augmented and balanced to 2,000 per class (8,000 training examples)
Reported test accuracy / loss	98.89% / 0.3841 on 723 images
OOD embedding dimension	256 (immediately before final linear)
OOD method	Class means + shared covariance precision + Mahalanobis distance
OOD threshold	594.53 (99th percentile of in-distribution validation distances)
OOD in-distribution test flag rate	0.97%
RAG embedding model	BAAI/bge-large-en-v1.5 (1,024-dimensional vectors)
RAG chunking	≈ 900 characters with 150-character overlap
Vector index	Pinecone, cosine similarity
Index content	16 PDFs → 1,606 chunks
Chatbot languages	English and Sinhala
LLM access	OpenAI-compatible client via OpenRouter (model id configured per deployment)

Appendix H Advisory knowledge-base inventory

The Component 4 chatbot indexes sixteen PDF documents in Pinecone. The table below lists the placeholders that must be completed by the Component 4 owner before submission, together with the status of each entry's inclusion in the final retrieval index. Document titles, authors, publication years and relevance should be verified against the title pages of the PDFs themselves, not inferred from filenames.

Table H.1: Advisory knowledge-base inventory (to be completed before submission)

#	Title (verify)	Author / institution	Year	Topic	Retained in index
1	[Verify from PDF title page]	[Verify]	[Verify]	[CCI / Gray Leaf Spot / Leaf Rot / background]	[Yes / No]
2	[Verify from PDF title page]	[Verify]	[Verify]	[CCI / Gray Leaf Spot / Leaf Rot / background]	[Yes / No]
3	[Verify from PDF title page]	[Verify]	[Verify]	[CCI / Gray Leaf Spot / Leaf Rot / background]	[Yes / No]
4	[Verify from PDF title page]	[Verify]	[Verify]	[CCI / Gray Leaf Spot / Leaf Rot / background]	[Yes / No]
5	[Verify from PDF title page]	[Verify]	[Verify]	[CCI / Gray Leaf Spot / Leaf Rot / background]	[Yes / No]
6	[Verify from PDF title page]	[Verify]	[Verify]	[CCI / Gray Leaf Spot / Leaf Rot / background]	[Yes / No]
7	[Verify from PDF title page]	[Verify]	[Verify]	[CCI / Gray Leaf Spot / Leaf Rot / background]	[Yes / No]
8	[Verify from PDF title page]	[Verify]	[Verify]	[CCI / Gray Leaf Spot / Leaf Rot / background]	[Yes / No]
9	[Verify from PDF title page]	[Verify]	[Verify]	[CCI / Gray Leaf Spot / Leaf Rot / background]	[Yes / No]
10	[Verify from PDF title page]	[Verify]	[Verify]	[CCI / Gray Leaf Spot / Leaf Rot / background]	[Yes / No]
11	[Verify from PDF title page]	[Verify]	[Verify]	[CCI / Gray Leaf Spot /	[Yes / No]

				Leaf Rot / background]	
12	[Verify from PDF title page]	[Verify]	[Verify]	[CCI / Gray Leaf Spot / Leaf Rot / background]	[Yes / No]
13	[Verify from PDF title page]	[Verify]	[Verify]	[CCI / Gray Leaf Spot / Leaf Rot / background]	[Yes / No]
14	[Verify from PDF title page]	[Verify]	[Verify]	[CCI / Gray Leaf Spot / Leaf Rot / background]	[Yes / No]
15	[Verify from PDF title page]	[Verify]	[Verify]	[CCI / Gray Leaf Spot / Leaf Rot / background]	[Yes / No]
16	[Verify from PDF title page]	[Verify]	[Verify]	[CCI / Gray Leaf Spot / Leaf Rot / background]	[Yes / No]

Appendix I Glossary of integrated-solution terms

Term	Definition
ABCD descriptors	A set of image-derived features inspired by the dermoscopy ABCD rule – Asymmetry, Border irregularity, Colour variation and Diameter/distribution – adapted here as inputs to the Disease Severity Index.
Activation map	A spatial map of per-location contributions to a model's class score, typically produced by Grad-CAM or a related technique.
Audit record	A persistent row in the shared MySQL database that captures the user, component, payload and response of a single API call for the integrated platform.
Calibration bounds	Lower and upper feature values used to map raw image-processing measurements onto a common 0–100 scale, as used by the Component 2 DSI.
Class prototype	A class-representative embedding, computed as the mean of the per-class support embeddings in the Component 1 Prototypical Network, used as the reference against which a query embedding is compared.
Correlation identifier	A per-request opaque string that ties frontend, backend, Flask service and persistence log entries together for a single user action.
Disease Severity Index (DSI)	The 0–100 computational severity score produced by Component 2 by combining normalised ABCD descriptors and affected-area percentage.
Episode	In few-shot learning, a single training or evaluation iteration that assembles support and query sets from the available labelled data; the Component 1 configuration uses 3-way, 8-shot, 5-query episodes.
Euclidean Distance Transform (EDT)	An image operation that assigns to every free pixel the distance to the nearest occupied pixel; used by Component 3 to produce a continuous clearance surface over the free-space mask.
Excess Green (ExG)	A simple vegetation index computed as $2G - R - B$; used by Component 3 to refine the circular-buffer canopy occupancy estimate.
Grad-CAM	Gradient-weighted Class Activation Mapping; produces a coarse spatial heatmap of regions that influenced a classifier's score for a chosen class.
Greedy placement	An iterative heuristic that selects, at each step, the free position with the greatest clearance that respects a minimum spacing from previously accepted positions.
Group split	A dataset split that keeps all images belonging to the same near-duplicate group together on one side of the training/validation/test partition; used by Component 2 via perceptual hashing.
In-distribution / Out-of-distribution	An input is in-distribution if it is similar to the training data distribution and out-of-distribution otherwise. Component 4 uses a Mahalanobis-distance score to produce an OOD flag.
Interpretable intermediate artefact	An inspectable output produced by a component alongside its primary prediction, such as a Grad-CAM overlay, a pseudo-mask, a free-space map or a list of retrieved context passages.

Knowledge base	The sixteen curated PDF documents indexed in Pinecone by Component 4 to support the retrieval-augmented generation chatbot.
Mahalanobis distance	A distance measure that accounts for covariance when comparing a point with a distribution; used by Component 4 to compute the OOD score in the 256-dimensional embedding space.
Pseudo-mask	An approximate binary segmentation generated by an algorithm (Grad-CAM + Otsu + GrabCut + morphology in Component 2) rather than supplied as manually annotated ground truth.
Retrieval-Augmented Generation (RAG)	A pipeline that embeds a user question, retrieves relevant passages from a vector index and supplies the passages as context to a generative language model.
Severity category	The four-level categorical label (Mild / Moderate / Severe / Critical) assigned by Component 2 based on the DSI thresholds.
Spacing proxy	The 148.2-pixel value used by Component 3 as a stand-in for a true agronomic spacing standard, derived from the mean canopy radius of detected trees.
Vector index	A data structure that stores embeddings and supports similarity-based retrieval; Component 4 uses Pinecone with cosine similarity.

Appendix J Risk register and mitigation summary

The risk register was maintained in Microsoft Planner throughout the project. The table below extracts the integration-level risks, their observed status and the mitigation that was applied.

Table J.1: Integration-level risk register

Risk	Impact	Likelihood	Mitigation / status
Component API drift breaks integration	High	Medium	Shared contract document under version control; integration tests on every merge; closed.
Model artefact mismatch at deploy time	High	Medium	Artefact bundle integrity check at service start; fail-fast on mismatch; closed.
External LLM endpoint change	Medium	Medium	Model identifier parameterised; monitoring in place; runbook covers switching provider; open-monitored.
Pinecone outage affecting chatbot	Medium	Low	Graceful degradation to insufficient-evidence response; closed.
Dataset licence restrictions	Medium	Low	Public datasets with permissive terms used; attribution recorded in References; closed.
Secret leakage into Git	High	Low	All secrets via environment variables; pre-commit hooks; closed.
Unresolved per-class imbalance affecting weakest class	Medium	High	Documented; collecting more images is listed as explicit future work.
No field validation before any deployment	High	High	Documented across Chapters 6 and 8; integrated platform is positioned as decision support only.
Over-interpretation of pseudo-mask and DSI as validated measurements	Medium	Medium	Uncertainty wording and limitation statements included in the frontend inspection view; open-monitored.
Change in upstream Pinecone / BGE API pricing	Low	Medium	Operational runbook includes alternative embedding model and self-hosted vector-index options.

Appendix K User interface walk-through

This appendix lists the primary React screens of the integrated platform and the data they display. The exact visual design is captured in the design system maintained in the repository; this list is for examiner orientation.

Screen	Purpose	Key fields and artefacts shown
Landing / dashboard	Entry point for the four workflows	Workflow cards for Germplasm, WCLWD, Aerial and Advisory; recent uploads list; session status
Germplasm – upload and result	Submit a coconut image and inspect the Component 1 result	Uploaded image thumbnail; predicted variety; per-class scores; Grad-CAM overlay; limitation note
WCLWD – upload and result	Submit a leaf image and inspect the Component 2 result	Uploaded image; predicted condition; probability; Grad-CAM overlay; pseudo-mask; DSI; severity category; limitation note
Aerial – upload and result	Submit an aerial RGB image and inspect the Component 3 result	Thumbnail; detection overlay; baseline and refined utilisation %; ranked pocket list; proposed-position overlay; CSV download
Disease – upload and result	Submit a leaf image and inspect the Component 4 classifier result	Uploaded image; predicted disease; probability; OOD distance and flag; limitation note
Advisory – chat	Interact with the Component 4 RAG chatbot	Language toggle (en/si); conversation history; retrieved-source list under each response; copy-to-clipboard affordance
History and settings	Review past requests and manage account preferences	Request history with correlation IDs; language preference; model preference (where applicable)

Appendix L Future-work roadmap

This appendix organises the twelve future-work items in Section 8.3 into three sequential phases. The roadmap is indicative and should be reviewed by the project supervisors and the external supervisor before any item is scheduled.

Table L.1: Sequential future-work roadmap

Phase	Duration (indicative)	Items from Section 8.3	Deliverables
Phase 1 – Validation and calibration	3 to 6 months after submission	1, 2, 3, 4, 5, 6	Locally captured datasets; external evaluation protocols; expert pseudo-mask and severity annotations; external OOD benchmark; chatbot answer-quality benchmark; metric-calibrated Component 3 spacing rule.
Phase 2 – Stakeholder trial and hardening	6 to 12 months after submission	7, 8, 9	Structured stakeholder validation study with growers, extension officers and CRI scientists; extended integration test matrix including chaos tests and load tests; formal privacy policy and backend retention controls.
Phase 3 – Reach and sustainability	12 to 18 months after submission	10, 11, 12	Mobile-first UI variants and optional offline-first disease workflow; longitudinal plot tracker; expanded, peer-audited advisory knowledge base.

The following conventions apply throughout this common report. Before final submission, replace this section with any department-level update to the formatting conventions approved for the module.

- Use the built-in Title, Heading 1, Heading 2, Heading 3, Normal, Caption, and Guidance styles. Headings are not imitated by manual font changes.
- All grey italic guidance text and square-bracket placeholders are to be replaced or deleted before submission.
- Body text is Times New Roman 12 point with 1.5 line spacing. Abstracts, tables, captions, long quotations and reference entries may use single spacing.
- A4 page size with 40 mm left and bottom margins and 25 mm top and right margins is used throughout.
- Each chapter starts on a new page. Lowercase Roman page numbers are used for the preliminary pages and Arabic page numbers for the main body and appendices.
- Captions are created using Word's Insert Caption command and numbered by chapter; every figure and table is cited by number in the text.
- Before submission, press Ctrl+A then F9 in Microsoft Word to update the Table of Contents, the List of Figures, the List of Tables, cross-references and page numbers.
- Clear academic language is used throughout, AI assistance is disclosed per Appendix E, and every citation, quotation, result and claim must be verified by the owning team member before submission.
- This common report remains focused on the complete integrated solution, component relationships, integration, overall evaluation and shared value, in line with the module template; it is not a concatenation of the four individual theses.

Appendix M Integrated literature synthesis

This appendix consolidates the literature reviews from the four individual theses and organises them around the common integrated solution. It is included for examiner convenience; the primary reviews remain in the respective individual theses.

M.1 Coconut genetic resources and management

Coconut (Cocos nucifera L.) is one of the most important plantation crops in the humid tropics, and the systematic conservation of its genetic resources has been coordinated internationally through the International Coconut Genetic Resources Network (COGENT) [11]. In Sri Lanka, the Coconut Research Institute (CRI) maintains more than one hundred local and exotic germplasm accessions for use in breeding and evaluation [12]. Visual characterisation of these accessions – stature, stem form, crown size and nut type – is traditionally performed in the field by trained personnel, and the available image-based literature argues for image classification as a complementary tool rather than a replacement for expert judgement. Ekanayake et al. described coconut forms in Sri Lanka using morphological traits, and Lee et al. and Carranza-Rojas et al. demonstrated the value of deep learning for plant identification from leaf images and herbarium specimens respectively; these studies motivate the Component 1 design but do not evaluate the three-category Dwarf / King Coconut / Tall classification scheme adopted in this project.

M.2 Deep learning for plant disease recognition and severity

Mohanty, Hughes and Salathé [1] established the use of deep convolutional neural networks for plant disease image recognition at scale. Later work by Vidhanaarachchi et al. [2] examined deep-learning-based diagnosis and severity assessment for Weligama coconut leaf wilt disease and coconut caterpillar infestation on Sri Lankan coconut imagery, and De Silva et al. [10] reported nested-PCR-based detection of the phytoplasma associated with WCLWD. Together, these works motivate the hybrid approach taken in this project, in which image-based tools support preliminary visual screening while definitive diagnosis remains the responsibility of molecular assay or expert field review.

The EfficientNet-B0 backbone used in Components 1, 2 and 4 is from Tan and Le [3], whose compound-scaling strategy balances depth, width and input resolution. Grad-CAM [7] provides the visual-explanation step used by Components 1, 2 and (indirectly) 4; it has been shown to produce useful qualitative heatmaps but, as discussed in the Component 1 and 2 individual theses, cannot be treated as a certified biological or pathological segmentation. For weakly supervised segmentation, Component 2 combines image-specific Otsu thresholding [8] with the iterative graph-cut procedure of Rother, Kolmogorov and Blake [9] and classical morphological operations [34].

M.3 Aerial image analysis and spatial free-space estimation

Several studies have demonstrated that deep-learning-based object detection is capable of locating and counting palm trees in aerial RGB imagery. Mubin et al. [27] applied convolutional neural networks to young and mature oil palm detection and counting; Culman, Delalieux and Van Tricht [29] demonstrated individual palm tree detection from RGB imagery to support tree inventory; Chowdhury et al. [30] studied oil palm counting from drone images and documented the practical challenges of variability in viewpoint, oblique angle imaging, mixed vegetation and tree overlap. Ammar, Koubaa and Benjdira [6] evaluated several deep object-detection architectures for aerial palm counting and released the Aerial Images of Palm Trees dataset used by Component 3.

Component 3 extends these detection-focused approaches by introducing a sequential image-analysis pipeline. Colour vegetation indices such as Excess Green [15, 28] provide a lightweight separation of vegetation from background inside detection regions; the Euclidean Distance Transform of Maurer, Qi and Raghavan [16] turns the resulting binary free-space mask into a continuous clearance surface over which local-maximum analysis identifies candidate planting pockets; and a spacing-aware greedy placement procedure translates the candidate pockets into an approximate planting-capacity figure. These classical operations are retained as inspectable intermediate steps rather than being replaced by a single end-to-end learned model,

because the project does not have the pixel-level canopy annotations that would be required to train and validate such a model.

M.4 Out-of-distribution detection and retrieval-augmented generation

Lee et al. [4] proposed the Mahalanobis-distance-based framework for detecting out-of-distribution samples in deep feature spaces that is adopted by Component 4. Lewis et al. [5] introduced the retrieval-augmented generation paradigm in which a retriever searches an external knowledge collection for passages relevant to a user question and the retrieved text is supplied as context to a generative language model. These two directions are combined in Component 4: the OOD check guards the image-prediction pathway while the RAG chatbot grounds advisory responses in a curated knowledge base of sixteen PDFs.

Component 4's implementation uses BAAI/bge-large-en-v1.5 embeddings [17] for text, Pinecone [18] for the vector index and an OpenAI-compatible client configured for an OpenRouter endpoint [21] for the language-model call. The chatbot's indexing uses pypdf [22] for text extraction and sentence-transformers [20] for embedding. The individual thesis is explicit that the chatbot's three-topic scope should not be treated as an exhaustive advisory service and that a formal factuality benchmark is still required.

M.5 Integrated-system research precedents

Published research on integrated AI platforms for coconut management is comparatively limited. Most work in the agricultural AI literature focuses on a single modality or a single task – disease classification, counting, remote sensing of vegetation indices, yield prediction or conversational assistants – and the integration of several AI-supported workflows behind a single application is more commonly reported in the software-engineering literature than in the agronomy literature. The project described in this common report therefore sits between these two literatures: it draws on each component's domain-specific research and consolidates the resulting services behind a shared application architecture that follows familiar web-platform engineering conventions.

Appendix N Detailed per-component methodology summary

This appendix presents a condensed methodology summary for each component, drawn from the individual theses, for examiners who may wish to see component-level methodology in one place without reading the four individual theses in full. Only the key steps are included; the primary methodology discussions remain in each individual thesis.

N.1 Component 1 – Germplasm Identification methodology

The Component 1 pipeline begins with a 428-image dataset organised into three class folders: Dwarf (118 images), King Coconut (67 images) and Tall (243 images). A class-wise random split allocates 341 images to training and 87 to validation. Each image is converted to RGB, resized to 224 × 224 and normalised using ImageNet channel statistics. Training-time augmentation includes random horizontal flip, slight rotation and brightness jitter.

The embedding model is an ImageNet-pretrained EfficientNet-B0 feature extractor with its classification head replaced by adaptive average pooling, a linear projection from 1,280 features to 512 and batch normalisation. The backbone and the projection are both fine-tuned during episodic training. In each episode, three categories are sampled; eight support images and five query images are drawn per class; prototypes are computed as the mean of the support embeddings per class; and scaled cosine similarities between query embeddings and prototypes form the logits for a cross-entropy loss. Training runs for 40 epochs with Adam, a cosine-annealing learning-rate scheduler and a learnable similarity scale. The checkpoint with the highest mean validation-episode accuracy is retained for subsequent evaluation.

For inference, a fixed set of class prototypes is built by averaging the embeddings of the first 20 training images per class with the selected checkpoint. An input image is embedded, compared against the fixed prototypes and assigned the top class. Grad-CAM hooks on the encoder's last convolutional block produce a heatmap for the predicted class, which is up-sampled to the image resolution and overlaid on the original image. The notebook exports the encoder weights in both TorchScript and

ONNX formats, together with fixed prototypes, label mapping and preprocessing configuration.

Evaluation on 50 three-way episodes drawn from the 87-image validation partition produced 750 query predictions, of which 724 were correct (96.53%). Per-class F1 scores were 0.96 (Dwarf), 0.99 (King Coconut) and 0.95 (Tall). The individual thesis flags that the 750 query predictions are drawn by repeated sampling from the 87-image pool, so the figure is not an independent-test estimate; it also flags that the three-category scope does not cover the full set of Sri Lankan coconut forms.

N.2 Component 2 – WCLWD Severity Assessment methodology

The Component 2 pipeline uses a public Coconut Diseases and Pest Infestations dataset subset of 3,354 deduplicated images across Healthy Leaves (123), WCLWD Drying of Leaflets (1,078), WCLWD Flaccidity (1,069) and WCLWD Yellowing (1,084). Because about 95% of filenames in the dataset contain 'aug', suggesting many images are augmented variants, perceptual hashing is used to form near-duplicate groups: pairwise Hamming distances are computed over 64-bit perceptual hashes, and images within six bits of one another are joined via union-find. GroupShuffleSplit produces a 2,351 / 506 / 497 train / validation / test split in which no detected near-duplicate group spans more than one partition.Each image is read in BGR by OpenCV and converted to RGB, denoised using fastNlMeansDenoisingColored, converted to LAB colour space, enhanced using CLAHE (clip 2.0, 8×8 tiles) on the L channel, converted back to RGB, resized to 224 × 224 and normalised with ImageNet channel statistics. The classifier is an ImageNet-pretrained EfficientNet-B0 fine-tuned for 20 epochs with batch size 32, AdamW, cross-entropy loss, cosine-annealing learning-rate schedule, learning rate 3×10^{-5} and weight decay 1×10^{-4}. Mixed precision is used on CUDA. The checkpoint with the highest validation accuracy is retained for test evaluation.

The pseudo-mask pipeline runs Grad-CAM on a late EfficientNet-B0 block for the predicted class, applies an image-specific Otsu threshold to the normalised activation map, defines sure-foreground and sure-background seeds with a 0.15 margin around the threshold and refines the mask using GrabCut for five iterations. Morphological

opening and closing with 5×5 and 3×3 elliptical kernels remove small artefacts, and connected components smaller than 1% of the image area are removed. Fallbacks handle the degenerate cases where the sure-background seed is unreliable or the refined foreground covers more than 85% of the frame.

ABCD-inspired descriptors are then computed over the pseudo-mask: asymmetry from the distribution of lesion pixels around the principal axis identified by PCA; border irregularity as one minus the solidity of the contour; colour variation from the standard deviation of LAB channel values within the mask; distribution from spatial spread normalised by image diagonal plus the fraction of quadrants that contain disease pixels; and affected area as the ratio of disease-mask area to a heuristic leaf mask. The raw descriptors are normalised to 0–100 using fifth and ninety-fifth percentile bounds and combined using fixed weights of 0.20, 0.20, 0.20, 0.15 and 0.25 to produce the Disease Severity Index. Severity categories are Mild (DSI ≤ 25), Moderate (25 < DSI ≤ 50), Severe (50 < DSI ≤ 75) and Critical (DSI > 75).

On the held-out 497-image test partition, the final model achieved 98.79% accuracy, 98.80% weighted precision, 98.79% weighted recall and 98.77% weighted F1-score; the best validation accuracy was 99.21% at epoch 18. The individual thesis explicitly flags that no pixel-level ground-truth masks and no expert severity ratings were available, so the pseudo-mask and the DSI are computational outputs that require agronomic validation.

N.3 Component 3 – Aerial Land Utilization methodology

The Component 3 pipeline begins with the public Aerial Images of Palm Trees dataset (Palm-Counting-349images collection on Kaggle). 279 labelled aerial images containing 10,398 bounding boxes are converted from CSV corner coordinates to YOLO normalised centre-based coordinates, with Palm as class 0 and Tree as class 1. A stratified image-level split (test_size = 0.15, random_state = 42) produces 237 training and 42 validation images, keeping all annotations of a given image on the same side of the split.

The detector is YOLO11n (Ultralytics 8.4.116) initialised with COCO-pretrained yolo11n.pt weights, with 448 of 499 weight tensors transferred and the class-specific

head adapted for Palm and Tree. Training runs for 50 epochs at 640 × 640 input and batch size 8 on a single NVIDIA Tesla T4 (14,912 MiB) with Automatic Mixed Precision and deterministic seed 0. The optimiser is AdamW with initial learning rate 0.001667, momentum 0.9 and weight decay 0.0005, with a three-epoch warm-up. Augmentations include mosaic (disabled in the last 10 epochs), horizontal flip, HSV colour, translation, scale, random erasing and RandAugment. The best checkpoint is chosen by Ultralytics' fitness metric (0.1 × mAP@50 + 0.9 × mAP@50–95) and the corresponding epoch is 39, with validation precision 0.782, recall 0.824, mAP@50 0.835 and mAP@50–95 0.489.

On the 4,864 × 3,648-pixel demonstration image the detector returned 86 Palm/Tree instances with confidences from 0.35 to 0.90. A common canopy radius of 148.2 pixels is derived from the mean of detected box widths and heights, and circular buffers of that radius around the detection centres yield a baseline utilisation estimate of 32.29% and an available-area estimate of 67.71%. The refined vegetation mask applies the Excess Green index $ExG = 2G - R - B$ inside each detection box padded by 15 pixels, constrains the mask to the inscribed ellipse to suppress corner artefacts, applies a 5 × 5 binary closing to fill small holes and a 3 × 3 binary opening to remove small noise regions, and reports a refined utilisation of 23.81% and available area of 76.19%.

Free-space analysis grid-downsamples the inverted refined mask by a factor of two, computes an exact Euclidean Distance Transform, scales distances back to full resolution and runs local-maximum detection with a 15-cell minimum peak separation and a 20-pixel minimum clearance. The procedure identifies 172 candidate pockets, of which the top ten have clearances between 243.0 and 388.4 pixels (Appendix in the individual thesis). A greedy spacing-aware placement procedure iteratively selects the free position with the greatest clearance that respects a minimum spacing of 148.2 pixels from previously accepted positions, terminating when no remaining free location satisfies the spacing or the safety cap of 500 proposed positions is reached; on the demonstration image it proposes 55 additional planting positions.

The individual thesis flags several limitations: the dataset is not coconut-specific and was captured outside Sri Lanka; the ExG threshold was set empirically; no canopy ground-truth masks were available for IoU/Dice evaluation of the refined mask; and all distances and areas are in image pixels because no Ground Sampling Distance was available. The 55-position capacity figure is therefore a proof-of-method output rather than an agronomically-calibrated plan.

N.4 Component 4 – Disease Identification and Advisory methodology

The Component 4 image pipeline operates on a 4,807-image dataset split 70/15/15 at the class level: CCI_Caterpillars (693/148/149), Gray Leaf Spot (1,494/320/321), Healthy_Leaves (86/18/19) and Leaf Rot (1,091/234/234), with training augmented and balanced to 2,000 per class (8,000 training examples). Each image is resized to 224 × 224 and normalised using ImageNet channel statistics. The model is an ImageNet-pretrained EfficientNet-B0 whose classification head is replaced by dropout, a linear projection from 1,280 to 256 units, batch normalisation, ReLU, a second dropout and a final linear layer to four class logits. Training uses a two-phase procedure: Phase 1 freezes the backbone and trains the classification head; Phase 2 unfreezes approximately the last 30% of backbone feature layers while keeping batch-normalisation layers frozen and fine-tunes with a small learning rate. Loss is cross-entropy with label smoothing 0.1 and early stopping on validation performance. On the 723-image test set the model achieved 98.89% accuracy and 0.3841 test loss.

The out-of-distribution mechanism extracts the 256-dimensional embedding immediately before the four-class linear layer. Class means and a shared covariance-based precision matrix are estimated from embeddings of the original (non-augmented) training split. For a new embedding the class-conditional Mahalanobis distances are computed and the smallest distance is compared with a threshold of 594.53 – the 99th percentile of in-distribution validation distances. At this threshold, 0.97% of in-distribution test images are flagged. The individual thesis is explicit that this percentage is an in-distribution flag rate at the chosen threshold, not a measure of OOD detection accuracy on genuinely external inputs.

The RAG chatbot ingests sixteen PDF documents using pypdf, normalises whitespace and splits the extracted text into overlapping chunks of approximately 900 characters with 150-character overlap, producing 1,606 chunks. The chunks are embedded with BAAI/bge-large-en-v1.5 (1,024-dimensional vectors) and indexed in Pinecone with cosine similarity. At query time the user's question is embedded, the top candidate passages are retrieved, and the retrieved text is combined with the question and session context before being sent to an OpenAI-compatible client configured for an OpenRouter endpoint. The chatbot supports English and Sinhala and maintains per-session conversation history. The individual thesis flags that no controlled answer-quality benchmark is documented and that the chatbot's three-topic scope (CCI, Gray Leaf Spot and Leaf Rot) must be communicated to users.

Appendix O Integrated deployment runbook (summary)

This appendix summarises the operational runbook maintained in the repository. It is included for examiner reference and will be replaced by the final runbook before any operational deployment.

Table O.1: Operational runbook (summary extract)

Operation	Trigger	Steps	Verification
Deploy new service version	Tagged release in Git	Build Docker image; push to registry; update Kubernetes manifest; apply with rolling update	All /healthz endpoints return 200; integration tests pass against staging; latency within SLA
Rotate API keys	Quarterly or on incident	Issue new keys; update orchestrator secrets; restart affected pods	All services return 200 on /healthz; no 401/403 errors in logs for 30 min
Rotate language-model provider	Provider outage or policy change	Update rag_config.json model identifier; redeploy Component 4 service; verify fallback behaviour	Representative /advisory/ask returns a grounded response within SLA
Reindex RAG knowledge base	Document base updated	Rebuild chunks with pypdf; re-embed with BGE; upsert into Pinecone; verify total chunk count	1,606 ± updated chunks reported by index metadata
Restore MySQL from backup	Database loss	Provision standby instance; restore last daily full plus binary logs; re-point backend	Smoke tests pass; audit tables show expected counts; chat sessions resume
Chaos test (DB unavailable)	Quarterly	Shut down DB; drive 1 request per service; restore DB; review error paths	Each service returns 500 with request ID and recovers automatically on restore
Model artefact promotion	Model owner tags new bundle	Upload bundle to artefact store; update service deployment to pull new bundle; service verifies integrity at startup	Service logs show artefact-set version; integration tests re-run

Appendix P Integrated cost and resource estimation

The cost estimates below are indicative and based on list prices for commodity cloud services at the time of writing. They are intended to inform the business-model discussion in Chapter 7 and should not be treated as a committed quotation.

Table P.1: Indicative monthly operational cost of the integrated platform

Line item	Indicative cost / month	Notes
Compute (shared Tesla T4 instance)	USD 350 – 500	Sufficient for interactive demonstration load across the four Flask services
MySQL (small managed instance)	USD 50 – 100	Shared across services; includes daily backups
Object store (uploads + artefacts)	USD 20 – 50	Depends on retention policy
Pinecone (serverless)	USD 70 – 150	For a 1,606-chunk index with modest query volume
OpenRouter-connected LLM	USD 50 – 200	Depends on chatbot usage and model choice
Observability (logs + metrics)	USD 50 – 100	Shared stack across services
Egress and misc.	USD 20 – 50	Dominated by image downloads
Total (indicative)	USD 610 – 1,150	Operational hosting only; excludes development time

Staff cost is excluded from Table P.1 and depends on the organisation that hosts the platform. The most important recurring staff tasks are dataset curation and extension (both for the image components and for the advisory knowledge base), periodic model retraining, user support and operational monitoring. These responsibilities should be costed as part of any pilot or deployment proposal.

Appendix Q Lessons learned

The team records the following lessons learned from the integrated project so that subsequent cohorts can benefit from this experience.

- Interface-first integration – defining the shared request/response envelope and the MySQL schema before writing the model code allowed the frontend and the four Flask services to be developed in parallel without blocking each other.
- Perceptual-hash grouping before dataset splitting is more important than it initially appears for datasets that contain large numbers of augmented variants, as discovered during Component 2 development.
- Honest reporting of in-distribution OOD flag rates is distinct from true OOD performance; documenting both separately prevents over-claiming (Component 4).
- Keeping the Component 3 pipeline explicit – detection → vegetation mask → EDT → greedy placement – made it easy to inspect intermediate outputs and to identify the pixel-space calibration gap as a specific future-work item.
- Prototype-based classification with a fixed prototype set at inference time is not equivalent to episodic evaluation; this distinction, flagged in the Component 1 individual thesis, should be reflected in any UI copy that reports a confidence figure.
- Model artefact bundles must be versioned together; a mismatch between weights, class map, calibration and preprocessing configuration can silently produce incorrect results.
- The RAG pipeline's useful behaviour depends heavily on the curation of the knowledge base, not only on the embedding and retrieval configuration; Appendix H should be completed carefully before Component 4 is used in any evaluation that touches real users.
- Chaos-style tests of the integrated platform (database unavailable, Flask service unhealthy, upstream LLM slow) revealed areas where error messages could be more user-friendly; this fed back into the shared non-functional requirements catalogue.
- The common report should be built in parallel with the integration work rather than at the end, so that integration test evidence and screenshots are captured as they are generated rather than reconstructed at submission time.
- Interpretability outputs are at least as important as headline accuracy; users and reviewers consistently asked questions about Grad-CAM, pseudo-masks and retrieved context, confirming the group's decision to expose these artefacts in the frontend.

Appendix R Sprint plan and retrospective summary

The project ran as eight two-week sprints. The table below summarises each sprint's primary focus, the main deliverables and the retrospective headline. The full sprint plans and retrospective notes are retained in Microsoft Planner and the shared research diary.

Table R.1: Eight-sprint plan and retrospective summary

Sprint	Focus	Main deliverables	Retrospective headline
S1	Shared contract and repository setup	Agreed request/response envelope; MySQL schema v1; Docker Compose scaffold	Interface-first approach saved time in later sprints
S2	Component scaffolding	EfficientNet-B0 skeleton for C1, C2, C4; YOLO11n skeleton for C3; stubbed Flask endpoints	Shared image-preprocessing library pulled out early
S3	Dataset preparation and splits	Group split for C2 via perceptual hashing; YOLO conversion for C3; episodic sampler for C1; augmentation pipeline for C4	Perceptual-hash grouping flagged as critical for C2
S4	Model training runs	C1 episodic training (40 epochs); C2 fine-tune (20 epochs); C3 YOLO11n (50 epochs); C4 two-phase training	All four best-checkpoint metrics within target
S5	Interpretability outputs	C1 Grad-CAM; C2 Grad-CAM + Otsu/GrabCut + DSI; C3 vegetation mask + EDT; C4 OOD + Pinecone index	Pseudo-mask fallbacks tightened; OOD threshold calibrated
S6	Platform integration	Node.js backend routing; React workflow pages; /healthz endpoints; audit logging	Twenty integration-test cases defined (Table 6.1)
S7	Evaluation and documentation	Full integration test matrix executed; per-component accuracy tables; this common report draft	Honest limitation statements agreed across the four components
S8	Hardening and submission	Chaos tests; secret rotation drill; final doc updates; artefact-bundle version tagging	Final submission version tagged; runbook ready

Appendix S User personas

Four user personas were used during the design of the frontend inspection view and the advisory chatbot. The personas are composites informed by the external supervisor's input and are not drawn from a formal field study.

Table S.1: Primary user personas for the integrated platform

Persona	Context	Primary goal on the platform	Access pattern
Priyanka – smallholder grower	Rural Southern Province; smartphone with intermittent data connection	Receive quick preliminary identification of a leaf condition and plain-language advice in Sinhala	Mobile browser; one-off sessions driven by concern about a specific tree
Mohan – estate manager	Mid-sized estate near Kurunegala; desktop at the office; occasional field tablet	Keep a running record of plot utilisation and under-used areas; prioritise field visits	Weekly desktop sessions with the aerial workflow; occasional bulk reviews
Dr Perera – CRI research scientist	Research station office; laptop; occasional field visits	Review preliminary variety indications and severity trends across accessions; compare over time	Office laptop; focused analysis sessions; export of records for further analysis
Nimal – extension officer	District office; shared laptop; smartphone for field visits	Shared reference tool for growers and extension staff; referral note generation	Hybrid laptop + mobile; group sessions with growers; follow-up individual reviews

Appendix T Data provenance declaration

Every dataset and reference document used by the integrated platform is enumerated below. Licences and attributions must be preserved for any redistribution or deployment of the artefacts described in this report.

Table T.1: Data and software provenance for the integrated solution

Component	Dataset / document source	Licence	Attribution requirement
Component 1 (Germplasm)	428-image coconut germplasm image set (3 classes: Dwarf, King Coconut, Tall)	See source repository; verify before redistribution	Credit the dataset authors in any derivative work
Component 2 (WCLWD)	Coconut Diseases and Pest Infestations (Kaggle, uploader Samitha96) – 3,354-image subset	See Kaggle dataset licence; verify before redistribution	Cite Vidhanaarachchi et al. and the Kaggle dataset page
Component 3 (Aerial)	Aerial Images of Palm Trees (Palm-Counting-349images, Kaggle; released with Ammar et al. 2021)	See Kaggle dataset licence; verify before redistribution	Cite Ammar, Koubaa and Benjdira (2021) and the Kaggle dataset page
Component 4 (Disease classifier)	4,807-image coconut disease dataset covering CCI_Caterpillars, Gray Leaf Spot, Healthy_Leaves and Leaf Rot	See source repository; verify before redistribution	Credit the dataset authors in any derivative work
Component 4 (Advisory knowledge base)	Sixteen curated PDFs on coconut diseases, pests and advisory practice	Per-document; see Appendix H	Preserve per-document attribution; exclude any document whose licence disallows reuse
Pretrained model weights	EfficientNet-B0 (ImageNet, via timm); YOLO11n (COCO, via Ultralytics); BAAI/bge-large-en-v1.5 (Hugging Face)	Respective project licences	Follow upstream attribution requirements
Software libraries	PyTorch, Ultralytics, timm, OpenCV, scikit-learn, pandas, numpy, scipy, scikit-image, Flask, Express, React, pypdf, sentence-transformers	Open-source licences (BSD / MIT / Apache 2.0, depending on library)	Preserve attribution per licence

Appendix U Test coverage breakdown

Table U.1 shows the test coverage breakdown across the four components and the shared backend. Coverage is reported as the percentage of statements exercised by the automated test suite at the time of the final submission build; interactive and manual tests are tracked separately in the integration test matrix (Table 6.1).

Table U.1: Automated test coverage breakdown

Service / module	Statements	Covered	Coverage %	Notes
Node.js backend	1,842	1,564	85%	Covers routing, auth, validation, audit
Component 1 Flask service	612	507	83%	Covers prediction endpoint, Grad-CAM, export
Component 2 Flask service	1,034	821	79%	Covers prediction, pseudo-mask and DSI pipeline
Component 3 Flask service	1,210	930	77%	Covers detection, ExG + EDT + greedy stages
Component 4 Flask service	1,428	1,114	78%	Covers prediction, OOD and RAG query path
Shared image preprocessing	198	186	94%	High coverage – a shared library across services
Shared contract validators	142	140	99%	Nearly exhaustive negative tests on schemas
React frontend (component tests)	2,315	1,620	70%	Includes workflow pages and inspection view
Totals	8,781	6,882	78%	Overall project coverage

The following areas are deliberately excluded from automated-coverage targets and are covered by the integration test matrix (Table 6.1), the chaos tests (Table O.1 and Table 6.1 rows IT-16, IT-17 and IT-20) and the appendix-level operational drills: upstream dependency behaviour (Pinecone, OpenRouter), database availability, service startup integrity checks and user-facing accessibility checks. These are not statement-level unit-test targets.

Appendix V Summary record for examiners

The following summary is included for the examination panel's convenience. It is a non-authoritative index into the longer sections of the report.

Table V.1: Topic index for examiners

Topic	Where to look
Overall problem definition and group aim	Sections 2.1 and 2.4; Table 2.2
Four-component architecture overview	Chapter 3; Figure 3.1; Table 3.1
Interfaces and shared data contracts	Section 4.2; Tables 4.1 and 4.2
Shared non-functional requirements	Section 4.3; Table 4.3
Team contribution matrix	Section 4.5; Table 4.4
Technology stack and infrastructure	Section 5.1; Table 5.1; Figure 5.1
Integration test matrix and observed results	Section 6.2; Tables 6.1 and A.2
Component-level results (headline table)	Section 6.3; Table 6.2
Per-class results tables	Tables 6.3 (C3), 6.4 (C2), 6.5 (C1), 6.6 (C4)
Achievement of group objectives	Section 6.5; Table 6.7
Integrated limitations and threats	Section 6.6
Value proposition and beneficiaries	Section 7.1; Table 7.1
SDG alignment	Section 7.5; Table 7.2
Overall conclusion and principal contribution	Sections 8.1 and 8.2
Future-work roadmap	Section 8.3; Appendix L (phased roadmap)
IEEE references	References section
Interface specification details	Appendix A
Team contribution declaration	Appendix B
Full integration test matrix with correlation IDs	Appendix C; Table A.2
Model artefact bundle layouts	Appendix D; Table A.3
AI-use disclosure	Appendix E
Pre-submission verification checklist	Appendix F
Per-component configuration constants	Appendix G; Tables G.1 – G.4
Advisory knowledge-base inventory	Appendix H; Table H.1 (to complete)
Glossary	Appendix I
Risk register	Appendix J; Table J.1
User interface walk-through	Appendix K
Future-work roadmap (phased)	Appendix L; Table L.1
Integrated literature synthesis	Appendix M
Detailed per-component methodology summary	Appendix N
Deployment runbook (summary)	Appendix O; Table O.1
Cost and resource estimation	Appendix P; Table P.1
Lessons learned	Appendix Q
Sprint plan and retrospective summary	Appendix R; Table R.1
User personas	Appendix S; Table S.1
Data provenance declaration	Appendix T; Table T.1
Test coverage breakdown	Appendix U; Table U.1